



CSC3067: Video Analysis and Machine Learning

Group Project – Face Detection

Team 23

Aleksandr Kruchinin (A.K.)	40392759
Li Siu Chun “Miku” (L.S.)	40392751
Maxim Rakhimov (M.R.)	40376248

2024-25 Semester 1 (2241_AUT)

Table of Contents

1. INTRODUCTION	4
2. FEATURES IMPLEMENTED	5
2.1 CROSS-VALIDATION	5
2.2 DATA AUGMENTATION	6
2.3 HISTOGRAM EQUALISATION	8
2.4 CONFIDENCE SCORE IN DETECTION	9
2.5 NON-MAXIMA SUPPRESSION (NMS)	10
2.6 MULTI-SCALE SLIDING WINDOW DETECTOR.....	12
3. EXPLANATION AND ANALYSIS OF CHOICES	13
3.1 BASIC EXPLANATION OF CHOICES IN FEATURE DESCRIPTORS	13
3.2 BASIC EXPLANATION OF CHOICES IN CLASSIFIERS.....	13
3.3 FURTHER ANALYSIS OF CHOICES.....	14
4. PERFORMANCE OF MODELS.....	18
4.1 RESULTS TABLE (SEE SECTION 4.2 FOR LEGEND).....	18
4.2 TABLE LEGEND	19
5. ANALYSIS OF RESULTS.....	20
5.1 BEST PERFORMERS	20
5.2 CLASSIFIER – 1-NN (1 NEAREST NEIGHBOUR) VS KNN	22
5.3 CLASSIFIER – KNN’S RANGE AND TYPICAL BEST VALUE	23
5.4 CLASSIFIER - STANDALONE DECISION TREE VS RANDOM FOREST.....	24
5.5 OPTIMAL VALUE FOR RANDOM FOREST/ADABOOST.....	25
5.6 CLASSIFIER – SVM.....	26
5.7 PRE-PROCESSING TECHNIQUE: HISTOGRAM EQUALISATION.....	27
5.8 FEATURE DESCRIPTOR - GABOR FEATURES.....	29
5.9 FEATURE DESCRIPTOR – LBP VS HOG	30
6. FURTHER TESTING AND ANALYSIS	31
6.1 TYPE I AND TYPE II ERRORS, RECALL AND PRECISION	31
6.2 LEAVE-ONE-OUT-CROSS-VALIDATION (LOOCV)	32
7. DETECTOR PHASE	35
7.1 LOWEST ACCURACY AT 50.83%: FULL IMAGE RAW PIXELS → 1-NN, HISTOGRAM EQUALISATION OFF, TRAINED ON ONLY ‘FACE_TRAIN.CDATASET’.....	36

7.2 HIGHEST ACCURACY AT 92.50%: HOG → KNN = 2, HISTOGRAM EQUALISATION ON, TRAINED ON ONLY 'FACE_TRAIN.CDATASET'	38
7.3 BEST INTUITIVELY BASED ON RANDOM TESTING: FULL IMAGE → MLP, 15000 NEURONS, 30 EPOCHS, HISTOGRAM EQUALISATION OFF, TRAINED ON 'FACE_ALL.CDATASET'	40
7.4 LOWEST ACCURACY AT 50.83%: FULL IMAGE RAW PIXELS → 1-NN, HISTOGRAM EQUALISATION OFF, TRAINED ON 'FACE_ALL.CDATASET'	42
7.5 HIGHEST ACCURACY AT 92.50%: HOG → KNN = 2, HISTOGRAM EQUALISATION ON, TRAINED ON 'FACE_ALL.CDATASET'	44
7.6 BEST INTUITIVELY BASED ON RANDOM TESTING: FULL IMAGE → MLP, 15000 NEURONS, 30 EPOCHS, HISTOGRAM EQUALISATION OFF, TRAINED ON 'FACE_ALL.CDATASET'	46
7.7 RECOGNITION RATES ACROSS MODELS IN SECTION 7.1 TO 7.6	48
8. CONCLUSION.....	49
9. APPENDIX.....	50
9.1 VERIFICATION OF WORKINGS IN FEATURE EXTRACTION	50

1. Introduction

Face detection is a critical component of many modern technological applications, including security systems, user authentication, and human-computer interaction. The ability to accurately identify and localize faces in images is essential for tasks such as facial recognition, emotion analysis, and crowd monitoring. This report details the development of a face detection model that involves performing, upon a provided set of labelled data, various feature extraction techniques, classification methods to train the model, and finally testing it upon a larger, unlabelled “detector” image. MATLAB is used to perform all the necessary steps.

The pipeline of this project initially involves a baseline without any feature extraction, by simply using the full “raw” image pixels, as a feature descriptor. After that, other feature extraction techniques are also experimented with, where key characteristics of faces, such as edges, textures, and gradients, are captured using descriptors including Local Binary Patterns (LBP), Histogram of Oriented Gradients (HOG) and Gabor filters. These descriptors effectively distil the visual information of an image into a simpler and more condensed format for machine learning.

Next, for every feature descriptor, various classifiers including K-Nearest Neighbour (KNN), Support Vector Machines (SVM), Random Forests, AdaBoost and Multi-Layer Perceptron (MLP), are used to identify patterns within the extracted features that correspond to facial structures. Cross-validation is then performed and the performance is noted.

Finally, once the model is trained, it is tested across a set of larger images. A multi-scale sliding window technique is employed to scan through the image at multiple scales, to ensure all possibilities of orientations and sizes of faces are captured. Non-maxima suppression (NMS) is then applied to render all overlapping detections into a best one. Confidence scores are also shown for each detection, showing how confident the model was in its detections (which, specifically for KNN models, had to be an additional feature implemented using Euclidean distance and number of neighbours).

2. Features Implemented

The following features in this section are implemented into the entire project pipeline, due to improved performance from initial testing, and kept constant due to brevity (or there would be too many following permutations to test).

2.1 Cross-validation

After training the model on the training dataset, the model is cross-validated using a test dataset which is also labelled, to produce True Positives, False Positive, True Negatives and False Negatives. Metrics such as Accuracy, Recall, Precision and F1 score can be calculated using this. But for brevity:

- Only Accuracy for all models will be recorded in a table in this report (see Section 4.1)
- Extra metrics, Type I and Type II errors will be evaluated only for certain models (see Section 6.1).

Training/Testing is around a 70/30 split. Other splits can also be experimented with by editing 'face_train.cdataset' and 'face_test.cdataset'. To be as comprehensive as possible, Leave-One-Out-Cross-Validation (LOOCV), which is a most extreme way of cross-validating, was performed (see Section 6.2), but only on 2 models due to brevity and to illustrate the possible differences in any given scenario.

2.2 Data Augmentation

By changing a parameter, 'augmented' available in the provided 'loadFaceImages.m' to 0 or 1, the training and test data set can be augmented in size, from 124 to 910 images by very slightly changing each image (e.g. mirroring, offsetting a little).



Figure 2.2a. Augmenting one image into 4 or more new images.



Figure 2.2b. More examples of augmented images inside dataset.

This was empirically tested using the raw pixels → KNN pipeline.

With Data Augmentation off:

```
Final Evaluation on Test Set with Best K = 9:  
TP: 16, FP: 1, TN: 9, FN: 4  
Accuracy: 83.33%  
Precision: 0.94  
Recall: 0.80  
F1 Score: 0.86
```

With Data Augmentation on:

```
Final Evaluation on Test Set with Best K = 40:  
TP: 190, FP: 12, TN: 28, FN: 10  
Accuracy: 90.83%  
Precision: 0.94  
Recall: 0.95  
F1 Score: 0.95
```

Note that in our KNN implementation, the algorithm scans through a set range of K from 1 to 200 for the best accuracy amongst all the Ks. So out of all Ks, the best accuracy with data augmentation on is higher than with data augmentation off.

This makes sense because to have more information to learn from is useful for the model, as would be for a human being. And hence, the model becomes more robust. What is interesting however is how by very slightly changing existing data at the digital level, more data can be generated to more robustly create the model, which can be extremely useful (e.g. limited/small dataset, costs resources to gather more original data), which costs virtually no time compared to actually trying to gather more original data.

Henceforth, all models were done with data augmentation “on” (augment = 1).

2.3 Histogram Equalisation

This form of pre-processing technique was chosen over other techniques because it can automatically be done on a per-image basis without manual thresholding or setting. It optimizes an image by trying to create the most well-rounded image based on lighting distribution over pixels, and hence the idea was to use it to bring out the most detail. If images were captured differently due to lighting, histogram equalisation also balances them out as long as the picture contains only the desired target (which is true, in the dataset).

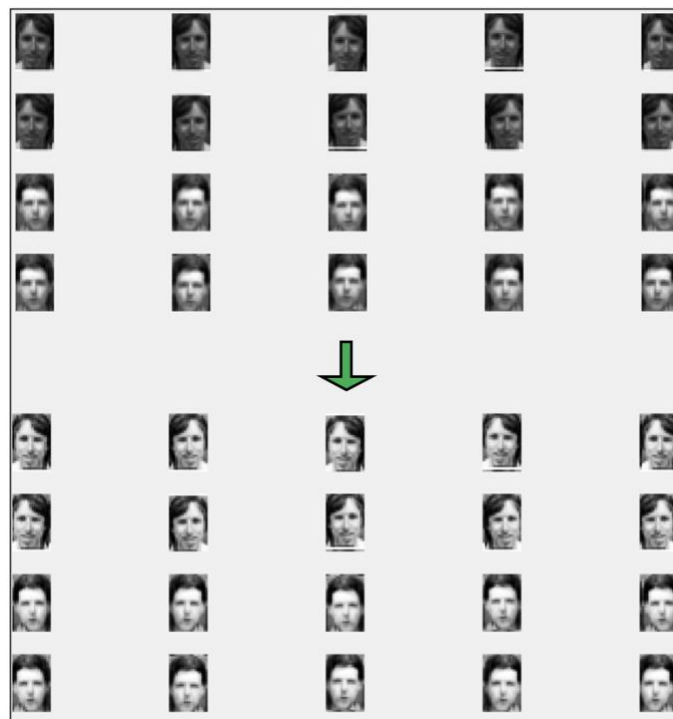


Figure 2.3a. Histogram equalisation balancing out the lighting differences within the dataset.



Figure 2.3a. Histogram equalisation bringing out much more detail at a glance.

Histogram equalisation can be applied in 3 different areas: the training dataset, the test dataset, and the final detector image. In this project, models are trained and tested in one go with histogram equalisation turned both on and off (i.e. on for both training and test, off for both training and test). On the other hand, regardless of how the model was trained, it was tested on a histogram equalized and non-histogram equalized final image.

Performance varied across the models during cross validation (see Section 4.1 and Section 5), but in general, histogram equalisation was found to be more detrimental than useful.

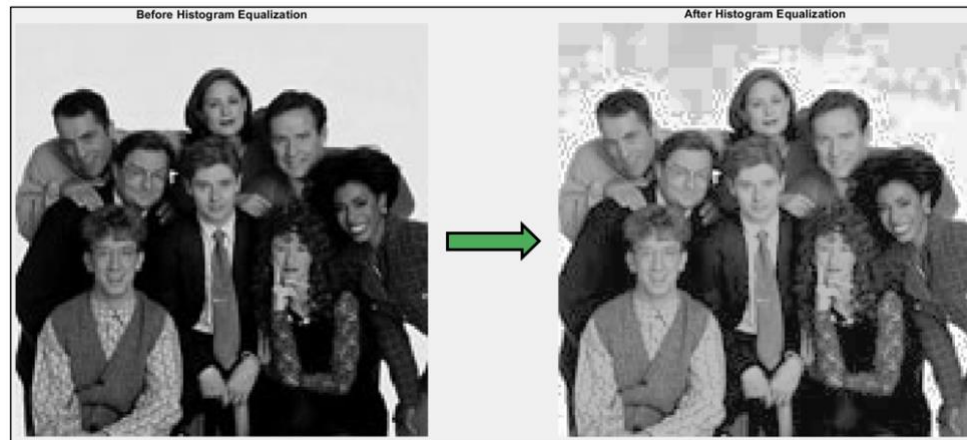


Figure 2.3c. Seen above, histogram equalization brings out a lot of detail (but can also be understood as noise, if a patch of the image was meant to be of a uniform color, like in the background). Additionally, the faces can be seen to become more washed out, and hence removes contrast within the face which a model usually uses to learn from, hence decreasing performance in general.

2.4 Confidence Score in Detection

While this was a built in feature in some classifiers, like SVM, some other classifiers like KNN did not inherently support a confidence score feature during its detection phase. Confidence in the detections made by KNN models was implemented by utilising the Euclidean distance between data points, and the distribution of labels for given Ks. By utilising these features, detections can be separated into confident ones where distances are shorter and the distribution of labels are more correct than not.

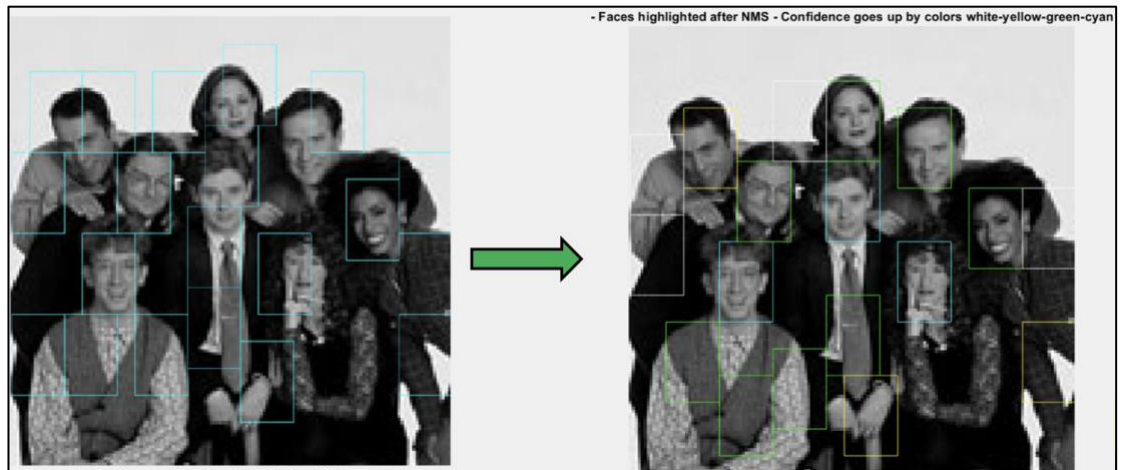


Figure 2.4a. Showing confidence scores in the detection boxes after implementation, marked by the colours white-yellow-green-cyan. Note how the real face detections tend to green or cyan colour, compared to the false positives.

Henceforth, confidence score during detection was applied to all models.

2.5 Non-Maxima Suppression (NMS)

In order to detect faces, a sliding window detector scans across horizontally across (at an increment of 6 pixels, to reduce the computational cost) the detector image from the top left corner to the top right corner until the right side of the window touches the end of the detector image, then goes back to its original position, then moves down a little (by 9 pixels, to reduce the computational cost) and scans across horizontally again until it reaches the end. The increments of 6 and 9 pixels were chosen since they were one-third of the original scale of the window (18x27 during training). A lower increment count can be used to increase number of detections but it would massively increase computational costs, and may be redundant since not a lot of new information is coming in at a time by such an already-low pixel increment (i.e., a face would generally not be missed out on by the detector if the window moves by one third of its size at a time).

Due to the nature of this sliding window detector though, there will be overlapping detections, especially near faces, when many detections exist for a single face due to slight variations. Since only one “best” detection is desired for one face, NMS is applied in order to select the best detection based on number of overlaps.

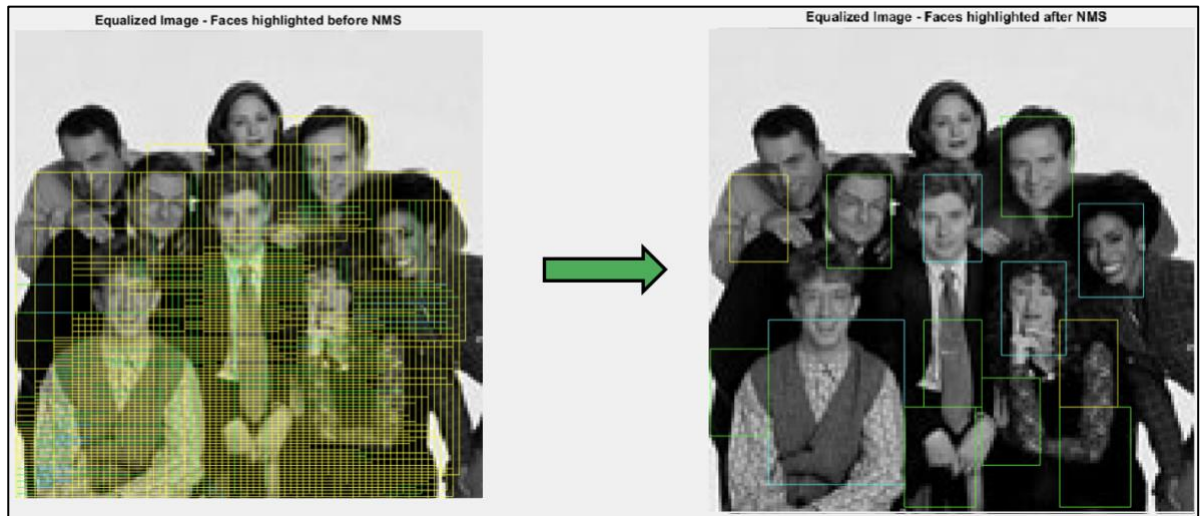


Figure 2.5a. Applying NMS to reduce the number of detections

It is important to note that for NMS, the threshold can be configured, and even at a value of '0.05', it was sometimes insufficient to remove overlaps. In general, this value can be as low as possible. In testing, by '0.01', no more overlaps were spotted in general during any models' detections.



Figure 2.5b. Showing a NMS threshold of 0.05 where overlaps still exist.

2.6 Multi-scale Sliding Window Detector

In order to detect faces at different sizes, which is especially a prevalent issue if the large detector image comes in different sizes, the sliding window detector must be adapted to be multi-scale instead of a fixed scale. To achieve this, after the end of a loop of scanning across an entire image completely, the detector window is increased by one pixel on each side, and the loop repeats until one of the axis on the window is as large as one of the axis the detector image. This increment can be increased or decreased, but 2 pixels on each axis is a sufficiently low enough number to be comprehensive, and is already computationally expensive.

```
Now using the model to detect in larger images using multi-scale sliding window.  
Processing scale [18x27] on original image: [ 25% 50% 75% 100%]  
Processing scale [18x27] on equalized image: [ 25% 50% 75% 100%]  
Processing scale [20x29] on original image: [ 25% 50% 75% 100%]  
Processing scale [20x29] on equalized image: [ 25% 50% 75% 100%]  
Processing scale [22x31] on original image: [ 25% 50% 75% 100%]  
Processing scale [22x31] on equalized image: [ 25% 50% 75% 100%]  
Processing scale [24x33] on original image: [ 25% 50% 75% 100%]  
Processing scale [24x33] on equalized image: [ 25% 50% 75% 100%]  
:  
Processing scale [136x145] on original image: [ 25% 50% 75% 100%]  
Processing scale [136x145] on equalized image: [ 25% 50% 75% 100%]  
Processing scale [138x147] on original image: [ 25% 50% 75% 100%]  
Processing scale [138x147] on equalized image: [ 25% 50% 75% 100%]  
Processing scale [140x149] on original image: [ 25% 50% 100%]  
Processing scale [140x149] on equalized image: [ 25% 50% 100%]
```

Figure 2.6a. Excerpt of the console output log in MATLAB during multi-scale sliding window processing of 'im3.jpg'

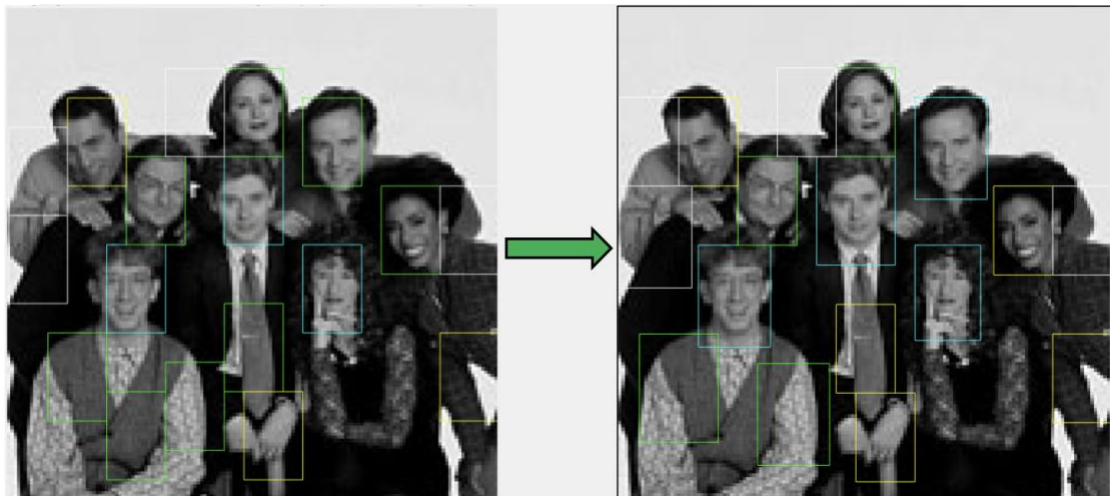


Figure 2.6b. After implementing multi-scale sliding window detector (right). Note how the boy with glasses is now captured better, as well as how the detection boxes vary slightly in sizes.

3. Explanation and Analysis of choices

The following explains why we chose the feature descriptors and classifiers that we did, and the interesting findings we made as we progressed. In general, the idea was to put into practice methodologies that were taught during lectures, as well as having a comprehensive enough list based on common and simple methods to demonstrate enough variability.

3.1 Basic Explanation of choices in Feature Descriptors

Benchmarking was a desired outcome:

- Full Image “Raw pixels” descriptor was chosen to be the baseline approach, since it is the most basic feature descriptor without any processing. It hence can show how much benefit feature extraction techniques can provide.
- PCA is meant to reduce dimensions, and it seemed logical that the most applicable feature set to reduce in dimensions should be the largest one, being full image raw pixels. PCA can then be compared with the baseline to see how much benefit it provides.
- LBP can be understood as a subset of HOG, since LBP performs neighbourhood comparisons in pixel intensities, and HOG takes it one step further by encoding directionality into these comparisons.

LBP, HOG and Gabor features are commonly employed in image detection, as they are simple techniques that work at a rudimentary yet crucial level.

3.2 Basic Explanation of choices in Classifiers

Coverage across learning approaches was a desired outcome:

- Instance-based: 1-NN and KNN directly rely on training instances for classification.
- Linear Classifier: SVM without a kernel provides a linear decision boundary.
- Ensemble Learning: AdaBoost and Random Forest combine multiple classifiers to improve performance.

- Tree-based: Standalone Decision Tree and Random Forest offer interpretable, hierarchical decision-making.
- Neural Network-based: MLP captures non-linear relationships and generalizes well with sufficient training data.

Benchmarking was also a desired outcome:

- By including both simple (1-NN, Standalone Decision Tree) and advanced (AdaBoost, Random Forest, MLP) classifiers, the project provides a comparison to identify the most effective approach for face detection based on efficiency and efficacy.
- Another comparison is between classifiers that are a subset of the other, for example 1-NN vs KNN, and Standalone Decision Tree vs Random Forest.

3.3 Further Analysis of choices

All feature descriptors were tested with every classifier.

For variables with changeable hyperparameters, 2 to 3 permutations were tested to illustrate differences but no more due to brevity. Namely:

- PCA can be applied to any feature descriptor to reduce their dimensions, but it was applied only to the Full Image “raw pixels” feature descriptor since it has the largest feature vector here (at 486, because of 18x27 pixels). PCA’s purpose is to reduce the feature vector down to the ones that preserve the maximum variance in the data, thereby retaining the most important information while improving computational efficiency and potentially reducing overfitting. The procedure for applying PCA is to first apply PCA on the training dataset to reduce it down to the specified dimensions. Then, using the trained eigenvectors extracted from this step, it is used to apply PCA separately to the testing dataset so it exists in the same realm of features as the training dataset, in order to not “cheat” on providing information to the training dataset during initial PCA dimensionality reduction.

- Since original dimensions of full image feature descriptor is 486, the final reductions of halving it down to 200, halving again to 100, and a more extreme scenario of 10 dimensions were chosen.
- It is interesting to note that attempting to try to select the final dimensions above 486 in the code is impossible, as expected, since it is not possible to increase dimensions above original count.
- Selecting a final value of 486 (i.e., applying PCA but not actually reducing any dimensions) slows down the computation. This is interesting because the dimensions are not being reduced but PCA computations likely still occur to try to do something, and slows down the program. This increase in execution time outweighs the time without PCA until dimensions are reduced down to around 70 dimensions. This could possibly be explained by how the program tries to run computations to find the best features for the desired final dimensions, which costs computational time and will become slower than if PCA was not used, but at a certain point where dimensions are low enough, the other computations are much simpler and hence the overall execution time becomes lower than without PCA.

```

18x27 = 486 dimensions
Elapsed time is 17.854936 seconds.
Final Evaluation on Test Set with Best K = 40:
TP: 190, FP: 12, TN: 28, FN: 10
Accuracy: 90.83%
Precision: 0.94
Recall: 0.95
F1 Score: 0.95

```

Figure 3.3a. Execution time of 17.85s without PCA.

3 dimensions	50 dimensions	100 dimensions	200 dimensions
Elapsed time is 7.644416 seconds.	Elapsed time is 12.907454 seconds.	Elapsed time is 25.454316 seconds.	Elapsed time is 36.142958 seconds.
Final Evaluation on Test Set with Best K = 35: TP: 190, FP: 20, TN: 20, FN: 10 Accuracy: 87.50% Precision: 0.90 Recall: 0.95 F1 Score: 0.93	Final Evaluation on Test Set with Best K = 78: TP: 200, FP: 18, TN: 22, FN: 0 Accuracy: 92.50% Precision: 0.92 Recall: 1.00 F1 Score: 0.96	Final Evaluation on Test Set with Best K = 61: TP: 190, FP: 12, TN: 28, FN: 10 Accuracy: 90.83% Precision: 0.94 Recall: 0.95 F1 Score: 0.95	Final Evaluation on Test Set with Best K = 40: TP: 190, FP: 12, TN: 28, FN: 10 Accuracy: 90.83% Precision: 0.94 Recall: 0.95 F1 Score: 0.95

Figure 3.3b. Execution times of 7.6, 12.9, 25.5 and 36.1s for respective 3, 50, 100 and 200 dimensions that PCA has reduced down to. Note how different values of K exists for different feature lengths.

- Histogram equalisation was the only pre-processing technique chosen, over others like Linear Stretching, due to how the intent was to bring out the most detail.
- SVM hyperparameters include Lambda (λ), Regularization Parameter (C), Kernel, and Kernel Option. A lot of different values were tested, but in general, Regularization Parameter had the most impact, and others flat-out did not work (like setting Kernel to 'gaussian').
 - Due to brevity as well, only C was adjusted, between '1' and 'inf'.
 - Other hyperparameters were kept at a suggested default of:

$$\lambda = 1^{-20}$$

$$\text{kernel} = \text{'poly'}$$

$$\text{kerneloption} = \text{'4'}$$
- Randomness of final model after each run of the program exists for the following classifiers:
 - Random Forest
 - MLP

This is due to how when these models generate and calibrate their architecture, there is randomness involved (like the layering in the trees of Random Forest, and the adjustment of weights in neurons of MLP). So on each execution of the program, the model generates slightly different models, and hence accuracies and final detections. The other classifiers however have fixed models due to how the calculations are discrete and always the same in obtaining the most optimal value. Hence all the values for these 2 classifiers are the averages across 5 runs.

- Random Forest and AdaBoost were tested, respectively, with 10, 100 and 200 trees/weak learners. '10' was chosen as a extreme scenario of a small value, and by '200' it was already very computationally expensive, as well as potentially overfitting, leading to '100' as a middle ground. '200' for both classifiers would take already around 10 minutes to run detections on 'im3.jpg'.
- MLP was permutated with number of total neurons and number of epochs. First with 1500 total neurons and 3 epochs, then changing it to 30 epochs, then to 15000 total neurons. Number of total neurons and epochs scaled noticeably with execution time. Seen below in Figure 3.3c, layers can be set up differently, but

having a Rectified Linear Unit (ReLU) layer after each hidden layer was found to have better results from brief testing. This is because it allows the network to model non-linear relationships, progressively extracting more abstract features at each layer, essential for tasks like image classification where images are composed of many different sets of elements.

```
layers = [  
    imageInputLayer(inputSize, 'Normalization', 'zscore')  
    fullyConnectedLayer(1000) % Hidden layer (adjust the number of neurons)  
    reluLayer  
    fullyConnectedLayer(500) % Another hidden layer (adjust the number of neurons)  
    reluLayer  
    fullyConnectedLayer(numClasses)  
    softmaxLayer  
    classificationLayer];  
  
% Set training options  
options = trainingOptions('adam', ...  
    'MaxEpochs', 30, ...  
    'MiniBatchSize', 32, ...
```

Figure 3.3c. How the architecture for the MLP looks, layer by layer, and hyperparameters at the bottom.

4. Performance of Models

4.1 Results Table (see Section 4.2 for legend)

Feature descriptor	Pre-processing techinque	Classifiers													
		1-NN	KNN	SVM		Standalone Decision Tree	Random Forest			AdaBoost			MLP		
				C=1	C=∞		10 Trees	100 Trees	200 Trees	10 Weak Learners	100 Weak Learners	200 Weak Learners	Epochs = 3 Total Neurons = 1500	Epochs = 30 Total Neurons = 1500	Epochs = 30 Total Neurons = 15000
Full "Raw" Image	None	50.83%	K = 40; 90.83%	70.83%	70.83%	67.50%	72.50%	76.25%	76.25%	80.83%	81.67%	81.67%	71.67%	73.75%	78.33%
	Histogram Equalization	69.17%	K = 67; 86.67%	72.50%	72.50%	63.75%	69.58%	77.92%	76.67%	76.67%	79.17%	78.75%	66.25%	74.17%	75.83%
Full "Raw" Image → PCA (10 Dimensions)	None	63.33%	K = 65; 90.00%	83.33%	57.50%	68.33%	72.92%	72.92%	72.50%	67.50%	70.00%	70.83%	55.83%	52.80%	54.17%
	Histogram Equalization	62.50%	K=89; 75.83%	16.67%	70.00%	75.00%	68.75%	83.75%	80.83%	80.00%	83.33%	83.33%	72.08%	66.67%	60.42%
Full "Raw" Image → PCA (100 Dimensions)	None	53.33%	K=61; 90.83%	65.83%	65.83%	72.50%	83.50%	86.25%	85.42%	80.00%	76.67%	75.83%	82.08%	77.50%	78.75%
	Histogram Equalization	70.00%	K = 71; 86.67%	72.50%	72.50%	78.33%	81.67%	83.75%	85.00%	83.33%	81.25%	83.33%	59.58%	71.67%	71.67%
Full "Raw" Image → PCA (200 Dimensions)	None	50.83%	K=40; 90.83%	71.67%	71.67%	72.50%	81.67%	86.67%	85.00%	70.83%	83.33%	78.33%	83.33%	77.08%	78.33%
	Histogram Equalization	68.33%	K = 67; 86.67%	71.67%	71.67%	76.67%	81.67%	85.00%	85.00%	80.00%	80.83%	83.33%	82.08%	74.17%	71.67%
LBP	None	63.33%	K = 73; 89.17%	87.50%	66.67%	57.50%	59.58%	69.17%	64.58%	65.83%	70.00%	64.17%	64.58%	61.25%	62.92%
	Histogram Equalization	63.33%	K = 59; 88.33%	86.67%	67.50%	50.00%	63.33%	67.08%	67.08%	63.75%	66.25%	67.08%	61.67%	60.00%	66.67%
HOG	None	87.08%	K = 2; 87.92%	84.17%	85.42%	60.42%	79.17%	77.92%	81.67%	76.67%	87.50%	87.50%	75.42%	82.08%	82.50%
	Histogram Equalization	91.67%	K = 2; 92.50%	82.50%	80.00%	70.83%	72.92%	80.00%	80.83%	76.25%	80.00%	80.00%	69.17%	72.19%	75.00%
Gabor	None	83.33%	83.33%	Broken	Broken	84.17%	83.33%	83.33%	83.33%	84.58%	84.58%	84.58%	83.33%	83.33%	83.33%
	Histogram Equalization	76.25%	K = 11; 85.42%	Broken	Broken	72.08%	78.33%	81.67%	83.75%	80.00%	82.08%	83.33%	72.50%	60.83%	72.19%

Table 4.1a. All the accuracies recorded for models permuted from the set of feature descriptors and classifiers.

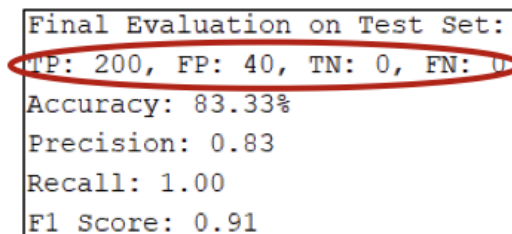
4.2 Table Legend

The model with the best accuracy across the whole table is marked with a green border

For each row, i.e. feature descriptor, the best accuracy it had with any classifier is colour coded in blue.

For each column, i.e. classifier, the best accuracy it had with any feature descriptor is colour coded in pink.

Cells coloured red denote that while an accuracy was produced, it was due to the model predicting everything during cross-validation as positive or negative for a face (this is why all red cells are 83.33% or 16.67%: because they were the result of 200 true positives and 40 false positives, or 40 true negatives and 200 false negatives). This phenomenon shows a broken model that only predicts either positive or negative without actually differentiating between its choices, making it an invalid model.



```
Final Evaluation on Test Set:
TP: 200, FP: 40, TN: 0, FN: 0
Accuracy: 83.33%
Precision: 0.83
Recall: 1.00
F1 Score: 0.91
```

Figure 4.2a. Models reporting this sort of results were invalidated.

Cells coloured orange denote a similar situation to the red cells, where the model predicted almost completely one-sidedly like above (maybe one to three predictions that were an exception). This was very obvious in the case of all classifiers using Gabor features.

Cells coloured grey denote that an invalid accuracy was produced due to failure during training of the model. This particularly only happened for Gabor → SVM. (See Section 5.8 for more information)

5. Analysis of Results

5.1 Best Performers

The model with the best performance across the board is HOG → KNN with Histogram Equalisation on, at 92.50% accuracy.

Across all the feature descriptors, the best classifier was definitively KNN (7 best accuracies out of 7).

- Classifiers such as SVM, Decision Tree, Random Forest, AdaBoost, and MLP showed varied performance depending on the feature descriptor. While these methods often excel when tuned or trained on optimal features, their performance can be hindered by sensitivity to incorrect hyper-parameterisation, feature scaling, noise, or overfitting in high-dimensional spaces. For example, MLP and SVM generally require more extensive pre-processing and parameter optimization to perform well, and Decision Tree-based models like Random Forest or AdaBoost can struggle with overly reduced features like PCA-10.

Across all the classifiers, the best feature descriptors were HOG (4 best accuracies) and Full Image “raw pixels” PCA-reduced to 200 dimensions (2 best accuracies) and 100 dimensions (1 best accuracy).

- HOG may have emerged as the best feature descriptor across all classifiers because it effectively captures structural and edge-based features, which are critical for tasks like image classification and recognition.
 - Unlike Full Image “raw pixels,” which can be sensitive to noise and variations in lighting, HOG focuses on gradients and orientations, making it robust to such changes and well-suited for extracting discriminative features like facial contours or object edges.
 - HOG may have also outperformed Gabor filters, since facial features may have more adequately been represented and learnt via edges (using HOG), instead of texture and frequency information (using Gabor).

- For Full Image PCA-reduced features, reducing the raw pixel dimensions to 200 helped achieve nominal accuracy by capturing useful patterns while discarding noise and redundant data.
 - Reducing it to 200 down from 486 dimensions boosted the accuracy likely because of how there are less features to distract the learning of the model that are irrelevant to capturing faces; basically that the feature set is now more focused.
 - However, as the dimensions decreased to 100 and then 10, the representations may have lost too many details, leading to successive drops in predictive power as a result of insufficient feature richness.

5.2 Classifier – 1-NN (1 Nearest Neighbour) vs KNN

KNN performed significantly better than 1-NN (1 Nearest Neighbour) across all the feature descriptors.

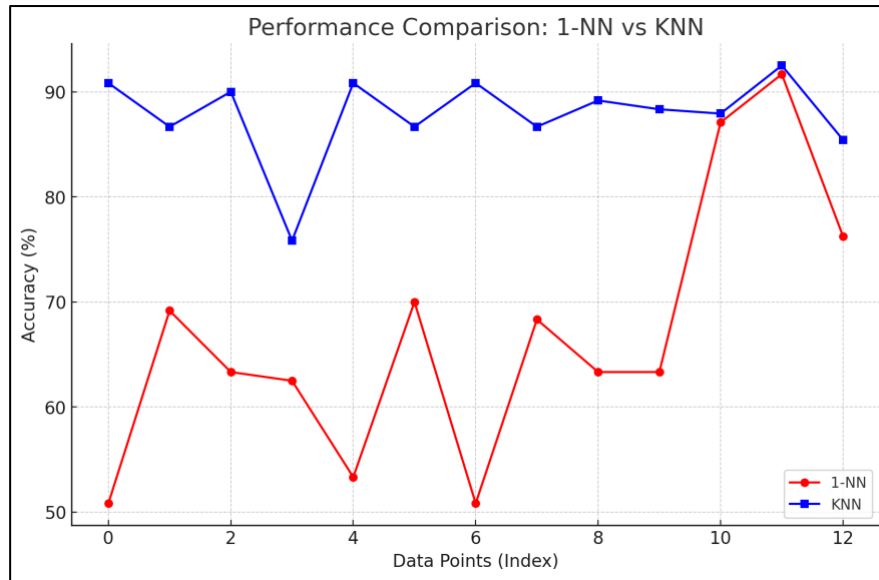


Figure 5.2a. Accuracy across all feature descriptors utilising 1-NN vs KNN (invalid Gabor set occluded).

1-NN performs worse than K-NN because it is highly sensitive to noise and outliers in the dataset. Since 1-NN relies solely on the closest training data point to classify a new sample, any mislabelled or noisy data point can lead to incorrect predictions. This sensitivity makes 1-NN less robust, particularly in datasets with complex distributions, overlapping classes, or outliers, as a single bad neighbour can heavily influence the decision boundary.

In contrast, KNN uses multiple nearest neighbours to make decisions, averaging their labels in classification. This voting mechanism reduces the influence of any single outlier or noisy data point, resulting in smoother and more reliable decision boundaries. By leveraging more data points, KNN becomes more reliable.

And in our pipeline, KNN always found for the best value of K across 1 to 200, so it would definitely have been higher anyways.

5.3 Classifier – KNN's range and typical best value

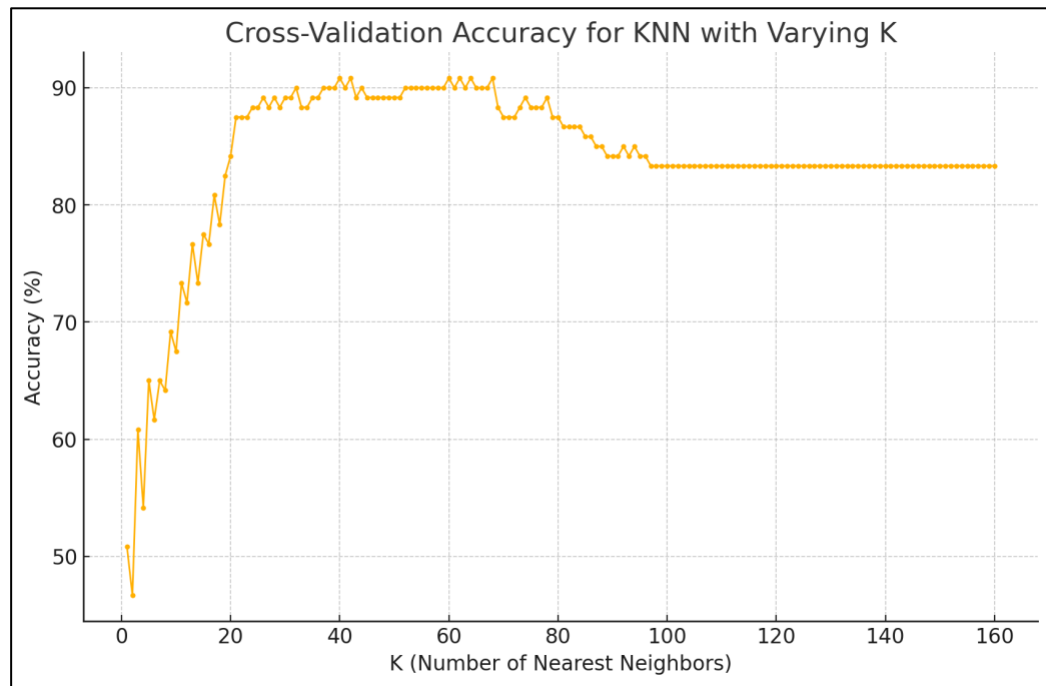


Figure 5.3a. General performance of KNN for most feature descriptors.

The picture above is for the raw pixels pipeline, but other feature descriptors showed similar curves. Note how KNN accuracy peaks between 20 to 100. Hence the max range to test for need not be much higher than that.

The shape of the curve reflects the trade-off between bias and variance. At very low Ks, accuracy is lower due to overfitting, as the model is highly sensitive to noise and outliers. As K increases, accuracy rises because the model smooths out local noise by considering more neighbours, reducing variance and improving predictions. This trend continues until around K=40, where accuracy peaks due to an optimal balance of local sensitivity and generalization.

Beyond that, accuracy begins to plateau and decline as the model starts to underfit. This is because higher K values incorporate neighbours from different classes, diluting local patterns and favouring the majority class. This behaviour leads to stagnation, particularly around $K > 100$, where predictions approach global majority-voting behaviour, reflecting the proportion of the dominant class in the dataset.

5.4 Classifier - Standalone Decision Tree vs Random Forest

Performance across the board for Standalone Decision Tree was worse compared to Random Forest in any given feature descriptor.

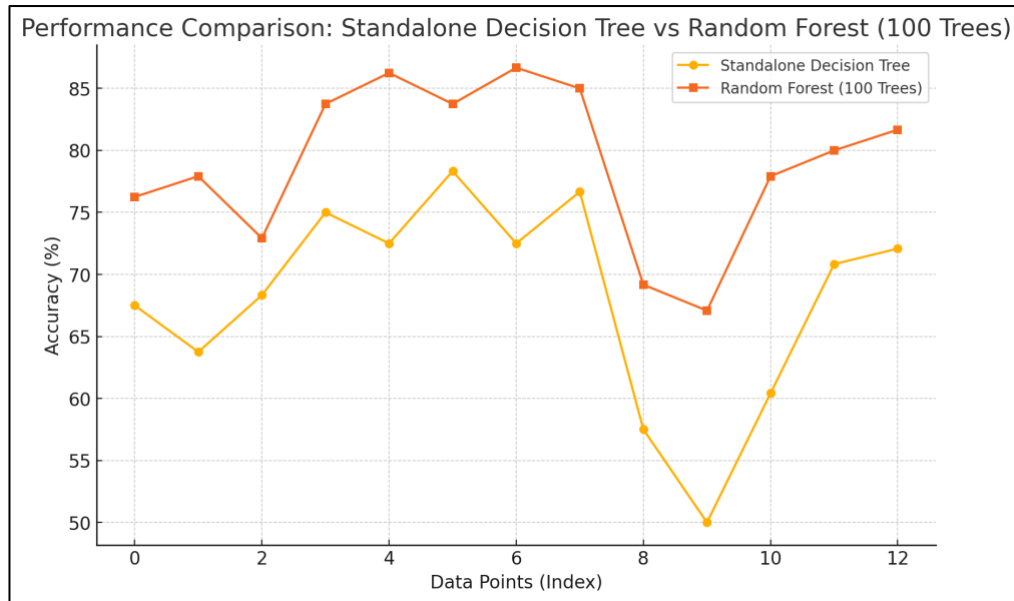


Figure 5.4a. Accuracies for all feature descriptors using Standalone Decision Tree vs Random Forest of 100 trees (invalid Gabor set occluded).

Since a Standalone Decision Tree is essentially only 1 tree in a Random Forest ensemble, it is prone to overfitting, especially when dealing with complex datasets. A single decision tree fully splits the training data to minimize errors, often creating a model that fits the training data too closely and fails to generalize well to unseen data. This issue is exacerbated when the dataset includes noisy or redundant features, as the standalone tree may create overly specific splits based on such features, leading to high variance and reduced predictive accuracy across feature descriptors.

In contrast, Random Forests mitigate overfitting by combining multiple decision trees, each trained on a random subset of the data and features. This ensemble approach reduces variance by averaging the predictions of the individual trees, resulting in more stable and robust performance. Random Forests are better at capturing general patterns in the data while ignoring noise, which is why they outperformed standalone decision

trees across all feature descriptors. The added diversity and robustness of Random Forests make them more effective classifiers, especially for datasets with high variability or limited training samples.

5.5 Optimal Value for Random Forest/AdaBoost

In general, the best accuracies were observed in 100 trees for Random Forest, and 100 weak learners for AdaBoost, which was the middle ground between 10 and 200.

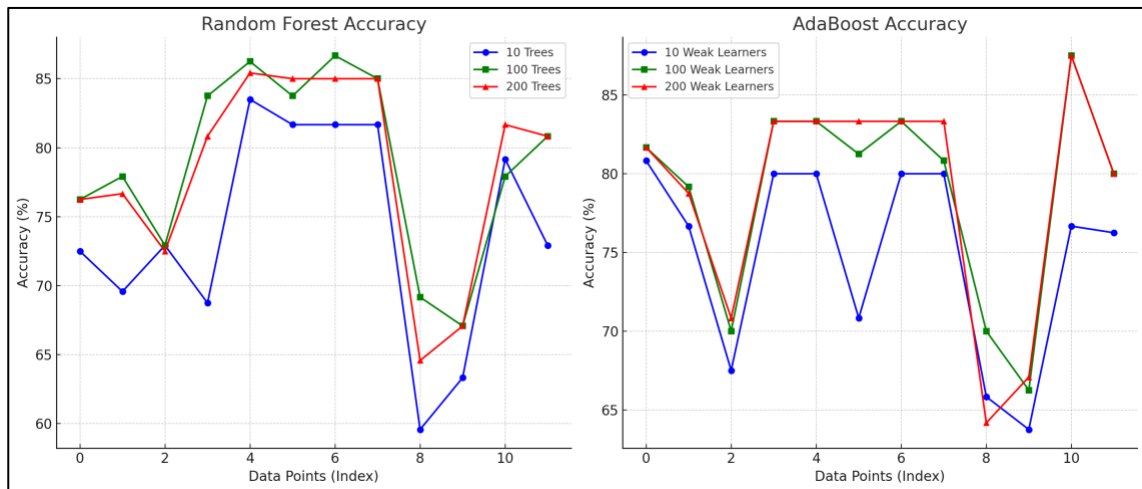


Figure 5.5a. Accuracy across all feature descriptors for Random Forest/AdaBoost utilising 10, 100 and 200 Trees/Weak learners (invalid Gabor set occluded).

This was likely due to the right balance between complexity and generalization. With only 10 trees/weak learners, both models underfit the data as they lack sufficient complexity to fully capture the underlying patterns. For Random Forest, 10 trees don't provide enough diversity or averaging to reduce variance effectively, while in AdaBoost, 10 weak learners aren't enough to iteratively correct enough misclassifications, leading to poorer performance.

And at 200 trees/weak learners, the models often face diminishing returns or even overfitting, especially on smaller or noisier datasets. In Random Forest, additional trees tend to replicate existing patterns, adding little new information while increasing computational cost. Similarly, AdaBoost with 200 weak learners places excessive focus on harder-to-classify samples, including noise or outliers, leading to overfitting and

reduced generalization. Execution time at this level of depth is also significant and not a desirable factor in real life, where processing things on a large scale with a long time may not be viable.

In contrast, 100 trees/weak learners provide an optimal balance. Random Forest achieves sufficient diversity to reduce variance without becoming redundant, while AdaBoost iteratively corrects misclassifications enough to improve accuracy without overfitting. This balance explains why both models perform best at this intermediate level.

5.6 Classifier – SVM

As a classifier, SVM seemed the hardest to make work. It is the only classifier with erroneous models barring the invalidated Gabor set. This may be due to incorrect hyperparameterization, but a lot of different values were tried to no avail.

0.67%	72.50%	72.5
=65; 0.00%	83.33%	57.5
=89; 0.83%	16.67%	70.0
=61; 0.20%	65.83%	65.8

Figure 5.6a. The cells in question under SVM, C=1 for PCA-10.

It is interesting to note that for PCA-10, SVM with a regularization parameter, C , of 1 produced erroneous results. This may be because using only 10 dimensions in the feature set may simplify the dataset and remove noise, but also risks losing critical discriminative information necessary for accurate classification; the data might lack sufficient variability, causing the SVM to struggle in finding an optimal hyperplane. This leads to either underfitting (if the hyperplane is too generalized) or poor margins (if the data clusters overlap significantly). With $C=1$, the SVM allows some misclassifications to avoid overfitting, but in such a low-dimensional space, the model may fail to correctly classify many samples. Hence, reduced feature richness combined with a moderately penalized regularization parameter may result in a fragile decision boundary that cannot generalize well, effectively "breaking" the model by making it unreliable for classification.

5.7 Pre-processing technique: Histogram Equalisation

While the idea of using this was to try to bring out more detail, in hindsight however, this worked made the results worse due to amplification of noise and unnecessary details (perhaps like eyewear or wrinkles which are not important features of defining a face).

Accuracies generally were worse with histogram equalisation on, with the exception of HOG utilising KNN, and validating all classifiers utilising Gabor features.

Another flaw that was noticed in our pipeline only at the end was that during the detector phase, the entire final image was histogram equalised. The correct technique would have been to only perform histogram equalisation within each sliding window detection while using the original image. This is due to how, like shown in Figure 2.3c, the background noise is amplified, because histogram equalization takes the whole image at once to process, including unnecessary parts of the image which are not used in the sliding window. This likely explained how the detections on a histogram equalised final detector image were usually worse.

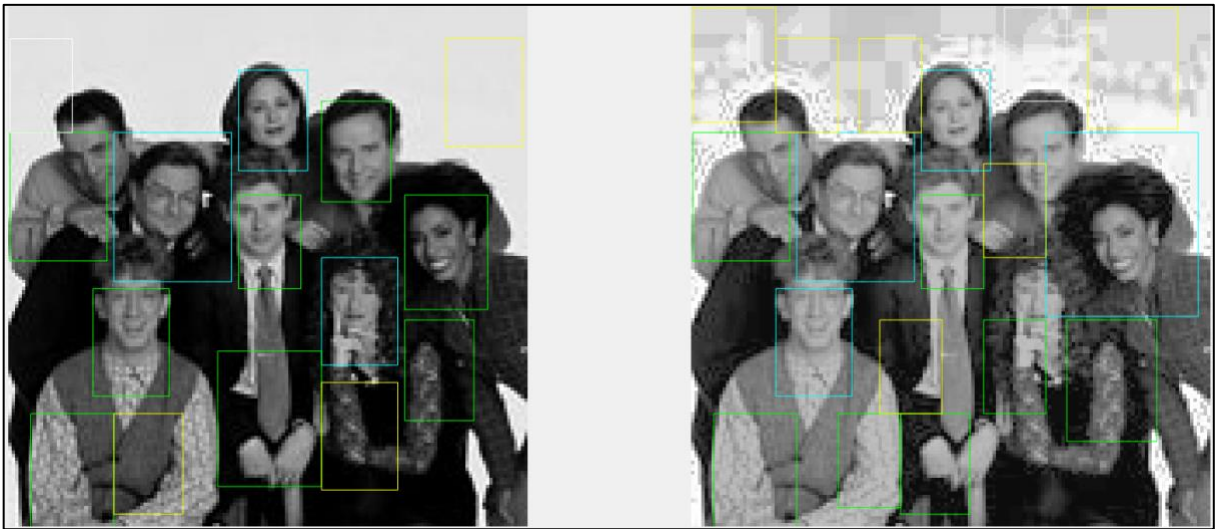


Figure 5.7a. Showing detections before and after the entire image being histogram equalised.

Note how the detections on the 3 rightmost people became worse likely due to incorrect technique usage.

In some specific cases however, histogram equalisation on the final image possibly boosted performance. In the following figure, the dark skin-toned woman on the right became more confidently detected by the model (seen by how the box went from yellow to cyan colour), albeit at a looser detection radius. This is likely due to how her face became lighter, and the model learnt from faces that had people with lighter skin tones than what she had in the original image. Similarly, however, other faces became too pale and the detections became worse, like the man right above her.

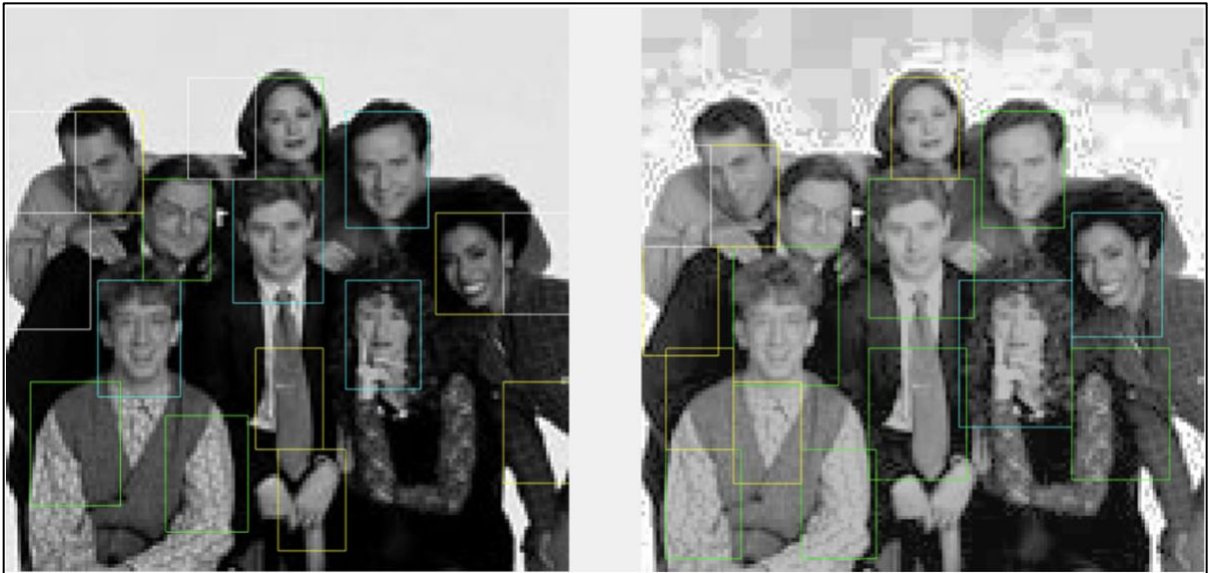


Figure 5.7b. Showing how performing histogram equalisation on the final image sometimes produced better results in specific areas, like the dark skin toned woman on the right.

5.8 Feature Descriptor - Gabor Features

As mentioned in Section 5.1, Gabor features may not be suited to face detections due to how it captures textures and frequency information. This likely caused the result of low accuracies across all classifiers.

What is notable is that virtually for all classifiers, with histogram equalisation off, all the models were invalid or almost invalid, but with histogram equalisation on, the models produced more reasonable results. This may be due to how with histogram equalisation applied, the dataset became more readable to the model with equalized lighting, in order to bring out better textures and frequency information.

Another notable thing is that SVM worked poorly with Gabor features. All such models predicted negative for everything. There was a unique warning output in the console during training phase, mentioning how the matrix is singular to working precision. Different hyperparameters were tried but nothing worked still.

Cost	Delta Cost	#support	#up	saturate
4.4761e-20	1.0000	1	0	
0.0000e+00	1.0000	2	0	
-1.5588e-20	0.0000	3	0	
-1.6014e-20	0.0273	4	0	
-6.7254e-18	99.9999	5	0	
-6.7254e-18	0.0000	6	0	
-7.0898e-18	0.0542	7	0	
-1.2869e-15	99.9999	8	0	
-1.2869e-15	0.0000	9	0	
-1.2869e-15	0.0000	10	0	
-1.2869e-15	0.0000	11	0	
-1.2869e-15	0.0000	12	0	
-1.2869e-15	0.0000	13	0	
-1.2869e-15	0.0000	14	0	
-1.2869e-15	0.0000	15	0	
-1.2869e-15	0.0000	16	0	
-1.2869e-15	0.0000	17	0	
-1.2869e-15	0.0000	18	0	
-1.2870e-15	0.0000	19	0	
Cost	Delta Cost	#support	#up	saturate
-1.2870e-15	0.0000	20	0	

Warning: Matrix is singular to working precision.
> In `monqpcinfy` (line 186)
In `svmclass` (line 130)
In `SVMTraining` (line 23)
In `Gabor_SVM_BROKEN` (line 57)

Figure 5.8a. Unique warning in the console output that did not happen for any other models, only Gabor → SVM.

5.9 Feature Descriptor – LBP vs HOG

As mentioned briefly in Section 3.1, since LBP is a simpler subset of HOG, it indeed reflects this in the results, where HOG consistently has a higher performance than LBP.

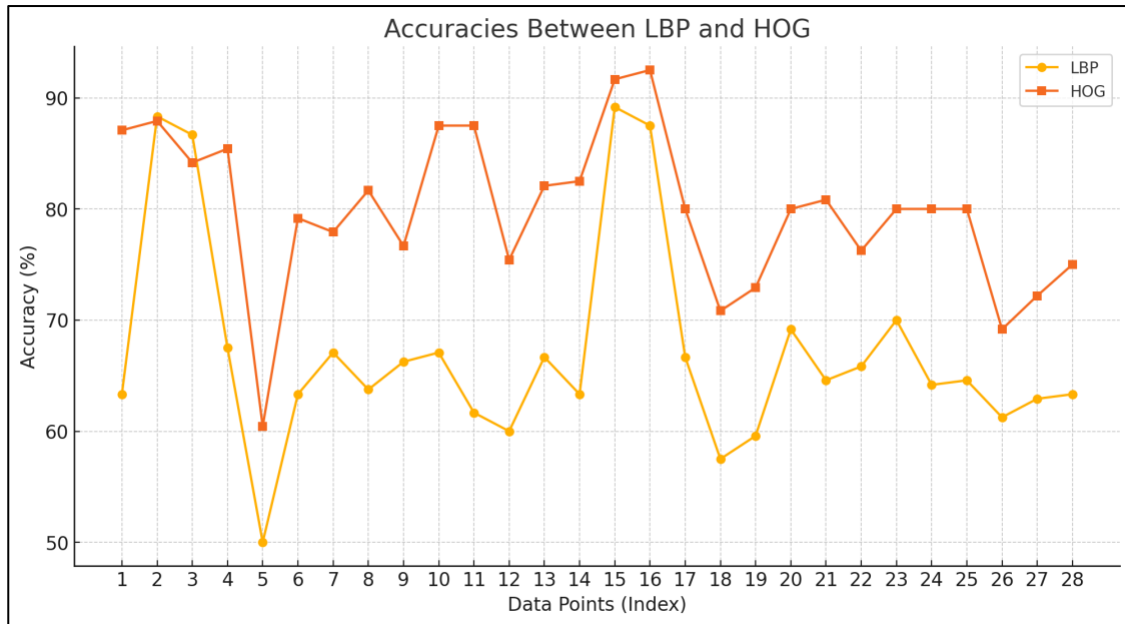


Figure 5.9a. Showing the accuracies measured across all the classifiers for LBP vs HOG.

This can be understood by how LBP is as a subset of HOG, where comparisons are limited to neighbouring pixel intensities, without explicitly storing the directional information of gradients. While LBP captures texture patterns by encoding pixel intensity differences as binary patterns, it does not inherently incorporate directional changes, which are key for recognizing edges or specific object boundaries. HOG, in contrast, uses gradient computation to capture the direction and strength of changes in intensity, which adds an additional layer of abstraction and generalization. This makes HOG particularly effective in scenarios where directional features (like the contour of a face) are crucial for classification. In essence, HOG's ability to be able to have directional information makes it better suited for tasks that require understanding shapes and patterns in a more global and structured way, while LBP remains a simpler method focused on local textural analysis.

6. Further Testing and Analysis

6.1 Type I and Type II Errors, Recall and Precision

From Table 3.1, it can be seen that models often shared the same accuracy. This gives a good reason for further metrics to be used, by analysing the True Positives (TP), False Positives (FP), True Negatives (TN), and False Negatives (FN). This analysis can be done for every model, as these metrics were produced for every model. However, for brevity, we will compare only 2 models, namely:

HOG → SVM, $C=\infty$, Histogram equalisation on

vs

HOG → AdaBoost, 100 Weak Learners, Histogram equalisation on

Both these models shared the same accuracy of 80.00%, but further metrics can reveal more.

1. HOG → SVM, $C=\infty$, Histogram equalisation on:

```
Final Evaluation on Test Set:
TP: 155, FP: 3, TN: 37, FN: 45
Accuracy: 80.00%
Precision: 0.98
Recall: 0.78
F1 Score: 0.87
```

2. HOG → AdaBoost, 100 Weak Learners, Histogram equalisation on:

```
Final Evaluation on Test Set:
TP: 162, FP: 10, TN: 30, FN: 38
Accuracy: 80.00%
Precision: 0.94
Recall: 0.81
F1 Score: 0.87
```

Despite having the same accuracy, which is $\frac{TP+TN}{Total\ predictions}$, the combination of these TP, FP, TN and FN may be different, resulting in other metrics, such as precision and recall, being different.

Type I errors (FP): The second dataset has a higher number of false positives (10 vs. 3), indicating that the 2nd model likely to mistakenly classify negative samples as positive.

Type II Errors (FN): The second dataset has fewer false negatives (38 vs. 45), indicating that the 2nd model is more likely to correctly identify positive samples, reducing the chance of missing true positive cases.

Between the two models, in the 1st model, precision ($\frac{TP}{TP+FP}$) is higher, whereas recall ($\frac{TP}{TP+FN}$) is higher in the 2nd model. This means that while the 2nd model undesirably captures more FP, it also desirably captures more TPs.

In other words, the 1st model may be suitable for scenarios where false positives have a high cost. For example, granting access into an area that requires clearance via facial scanning, and that a security breach from someone who is unauthorized (FP) would be unacceptable. Meanwhile, the lowered recognition rate for truly authorized personnel (TP) can be manually verified through other means with some extra time instead.

The 2nd model may be more suitable where missing true positives has a high cost. For example, a surveillance camera scanning through a crowd to try to catch wanted a criminal's identities, where an operator would not mind manually looking through more false positives for a higher chance at catching them (TP), and that if they were to slip away undetected (FN), it would be dangerous.

6.2 Leave-One-Out-Cross-Validation (LOOCV)

LOOCV can be performed for all models, once again, just like the analysis above in Section 6.1. However, for brevity, we will only perform LOOCV for 2 of the models to illustrate possible differences in any given scenario.

To perform it, the loading pipeline must first be adjusted. A new `.cdataset`, named 'face_all.cdataset' was created list all of the 124 images of just 94 previously in 'face_train.cdataset'. Next, new code was written to perform LOOCV instead of testing the model on a split dataset (see Figure 9.1f).

The 2 models to be examined are:

Full Image → PCA-100 → KNN, Histogram equalisation off

vs

HOG → KNN, Histogram equalisation on

And to take things one step further, again, LOOCV will take place for every single one of the Ks, in each model. That is to say, if the range of K is specified between 1 and 100, then LOOCV happens for K = 1 (910 folds, since there is 910 images) and the average accuracy is then calculated for this K, then it proceeds to do the same for K = 2, etc, until the max K specified (see Figure 9.1g). This is repeated for each model above.

1. Full Image → PCA-100 → KNN, Histogram equalisation off:

Best K = 2 with Average LOOCV Accuracy = 98.68%

2. HOG → KNN, Histogram equalisation on:

Best K = 2 with Average LOOCV Accuracy = 91.32%

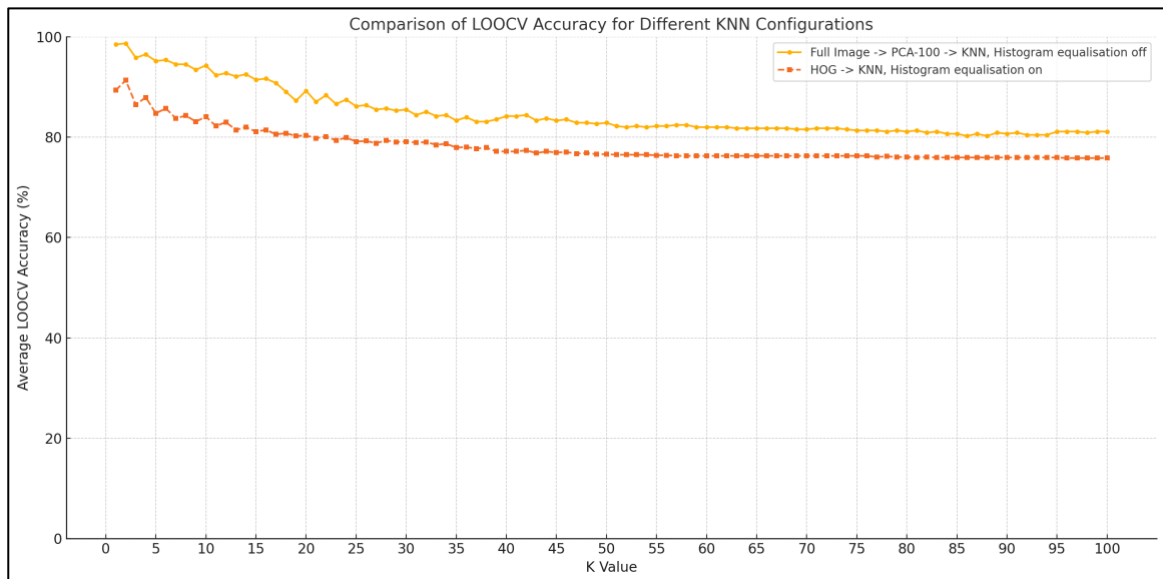


Figure 6.2a. Plotting the average LOOCV accuracy between the 2 KNN models, for K from 1 to 100.

It is interesting to note, from this observation, how firstly, LOOCV can (and should) produce a much higher accuracy in general compared to a split dataset, like in the case of the 1st model (previous accuracy from standard split was 90.83%). This is possibly because the model now has much more information to learn from each time (909 images to learn from during each fold), instead of the ~70/30% split resulting in only 670 images to learn from.

Secondly the 2nd model, while originally having a higher accuracy of 92.50% from the ~70/30% split test, now had a lower LOOCV average accuracy of 91.32%. This is peculiar and it is uncertain why, based on the aforementioned logic.

Overall, it reflects how using a different split between training/testing dataset, or in the extreme case of LOOCV, different accuracies can be produced, because a different model emerges altogether when given different items to learn from. In general, a higher accuracy should be produced. This is important going into the detector phase in Section 7, where the difference in a model's performance trained on a full dataset can be observed.

7. Detector Phase

The 4 final detector images ‘im{1|2|3|4}.jpg’ (non-histogram equalised), are tested on using various models, chosen for certain properties below.

Recognition rate will be under each figure. Faces that are well captured score one out of one point, faces that are partially captured score half a point.

Sections 7.1 to 7.3 are trained only on the original 70/30 split, using ‘face_train.cdaset’.

Sections 7.4 to 7.6 are trained on the full dataset of ‘face_all.cdaset’.

7.1 Lowest Accuracy at 50.83%: Full Image Raw Pixels → 1-NN, Histogram Equalisation off, Trained on only ‘face_train.cdataset’

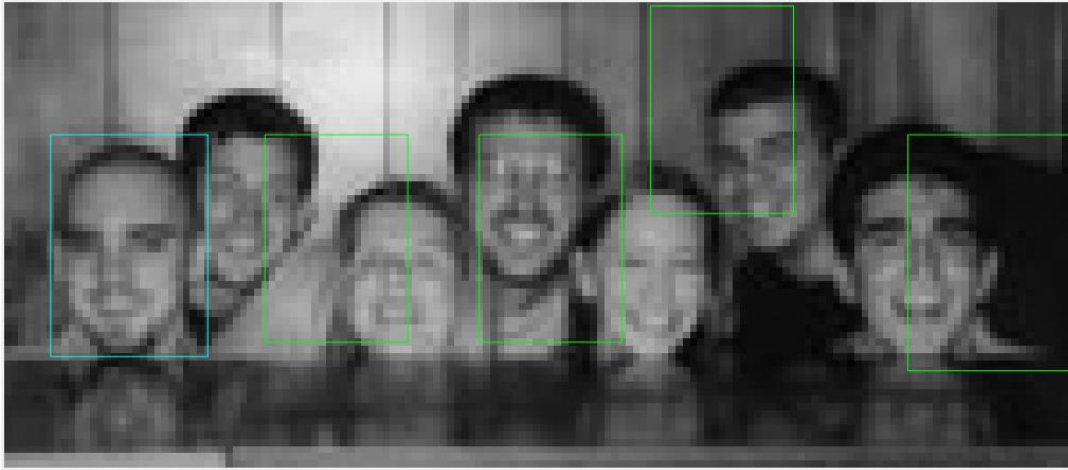


Figure 7.1a. im1.jpg (Recognition rate is 50%)

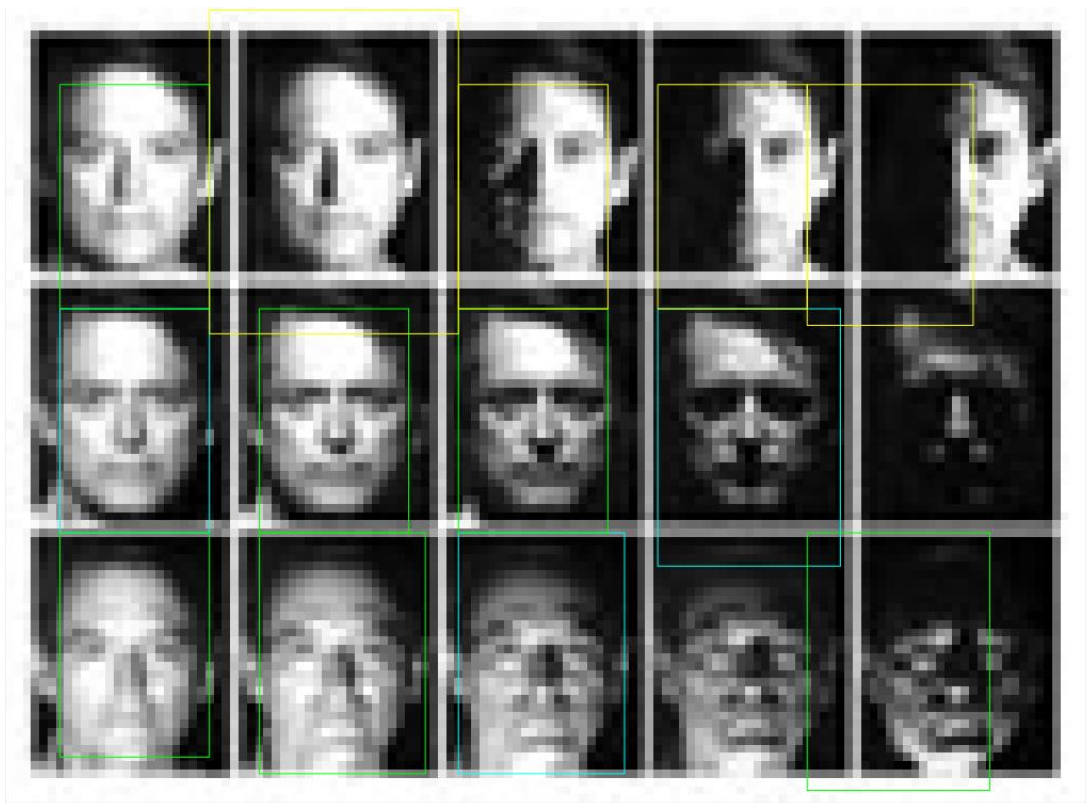


Figure 7.1b. im2.jpg (Recognition rate is 80%)



Figure 7.1c. im3.jpg (Recognition rate is 68.75%)

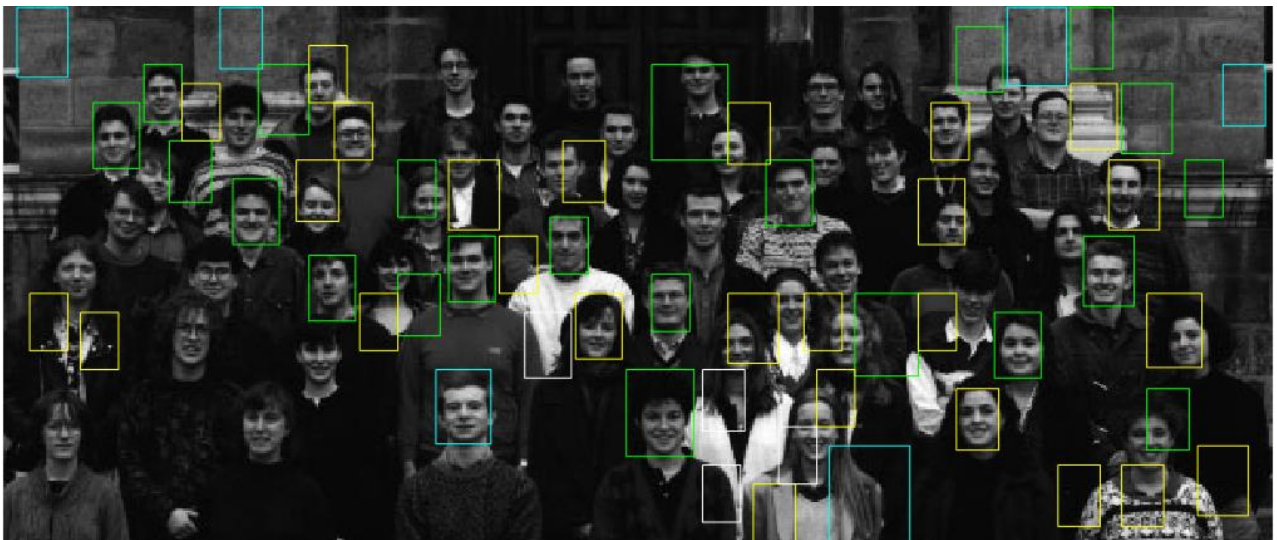


Figure 7.1d. im4.jpg (Recognition rate is 37.9%)

The average recognition rate of this model is 59.16%.

**7.2 Highest Accuracy at 92.50%: HOG → KNN = 2, Histogram Equalisation on,
Trained on only 'face_train.cdaset'**

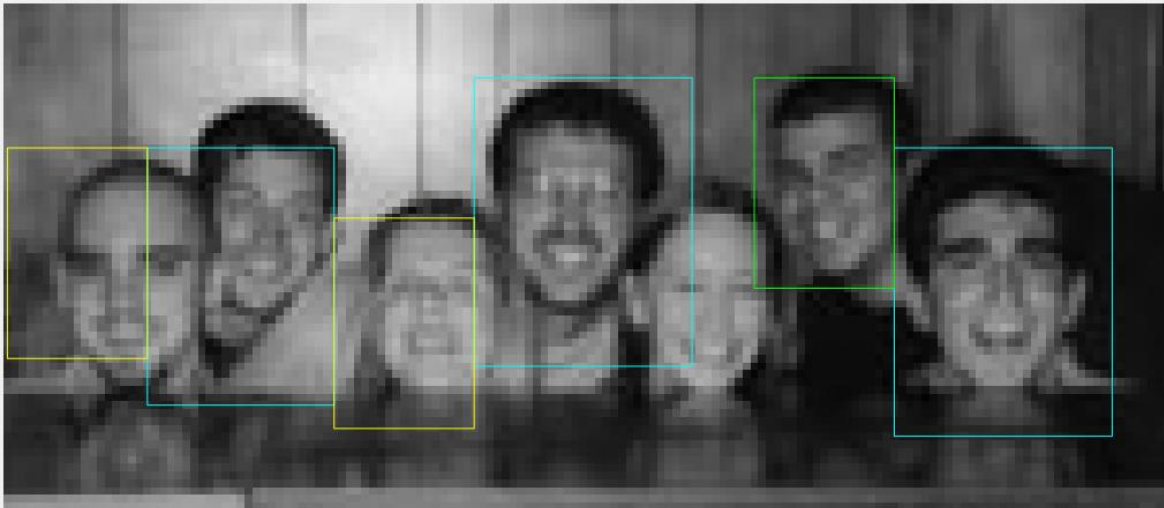


Figure 7.2a. im1.jpg (Recognition rate of 78.6%)

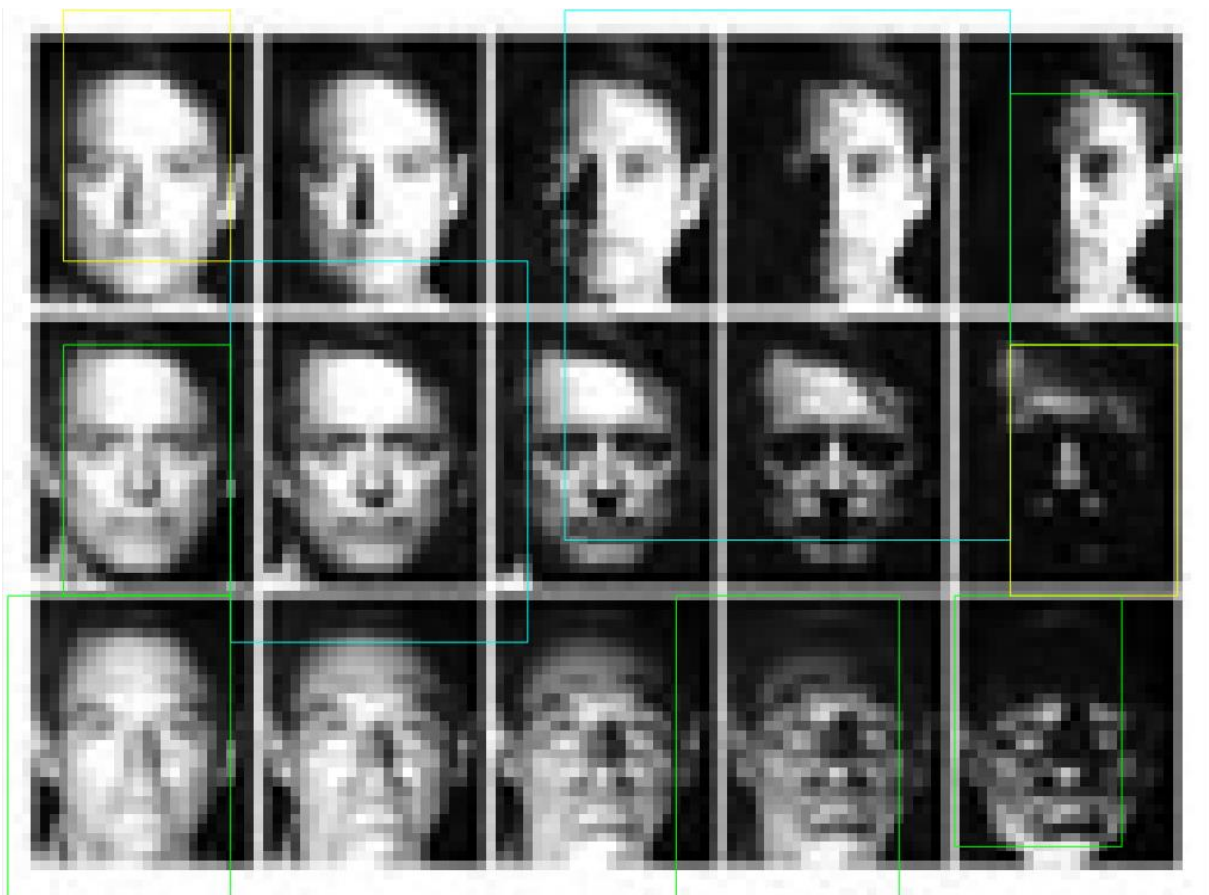


Figure 7.2b. im2.jpg (Recognition rate of 53.3%)



Figure 7.2c. im3.jpg (Recognition rate of 6.25%)

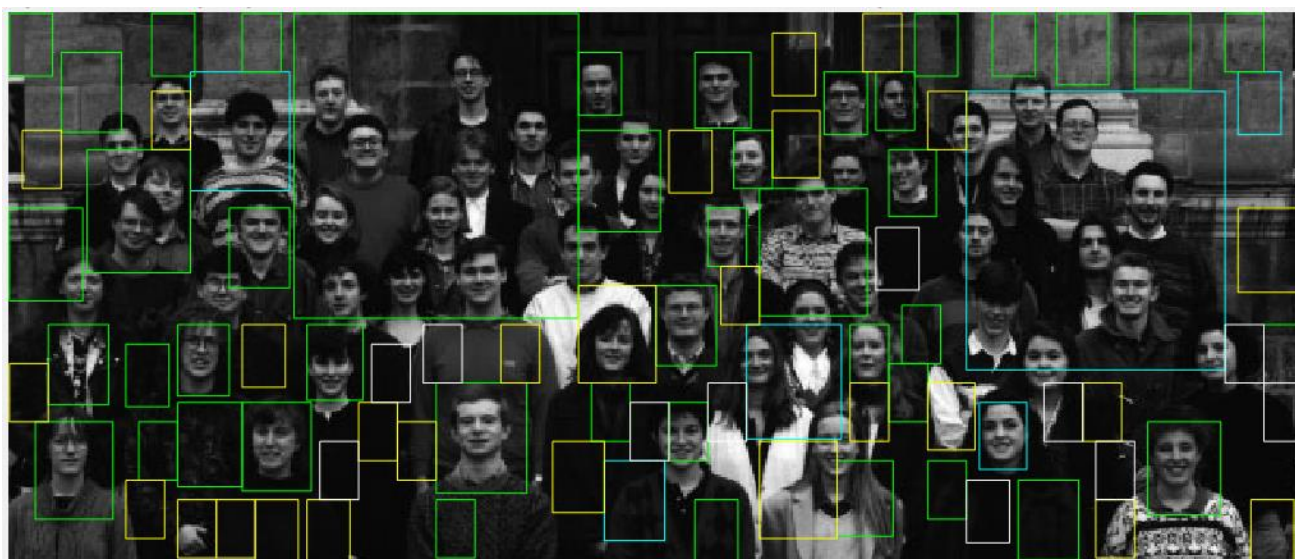


Figure 7.2d. im4.jpg (Recognition rate of 37.9%)

The average recognition rate for this model was 44%.

7.3 Best Intuitively based on Random Testing: Full Image → MLP, 15000 neurons, 30 epochs, Histogram Equalisation off, Trained on ‘face_train.cdaset’

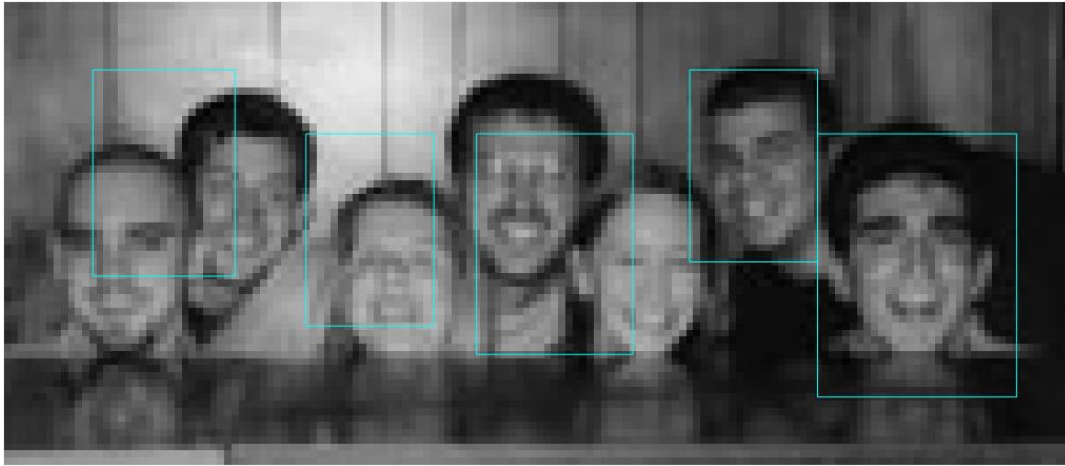


Figure 7.3a. im1.jpg (Recognition rate is 64.3%)



Figure 7.3b. im2.jpg (Recognition rate is 20%)



Figure 7.3c. im3.jpg (Recognition rate is 75%)

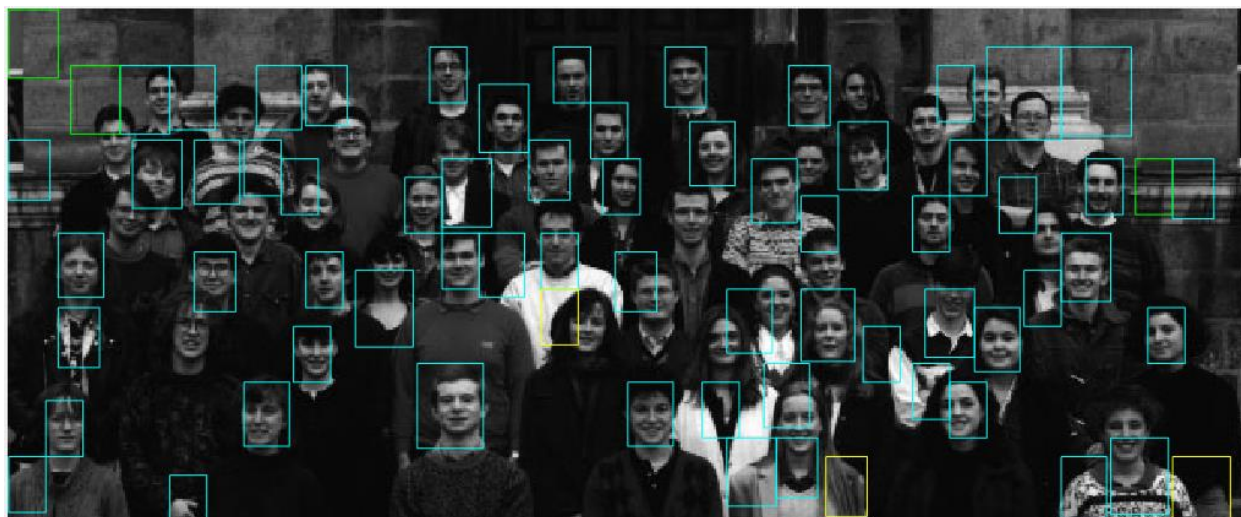


Figure 7.3d. im4.jpg (Recognition rate is 61.2%)

The average recognition rate for this model is 55.1%

7.4 Lowest Accuracy at 50.83%: Full Image Raw Pixels → 1-NN, Histogram Equalisation off, Trained on 'face_all.cdaset'

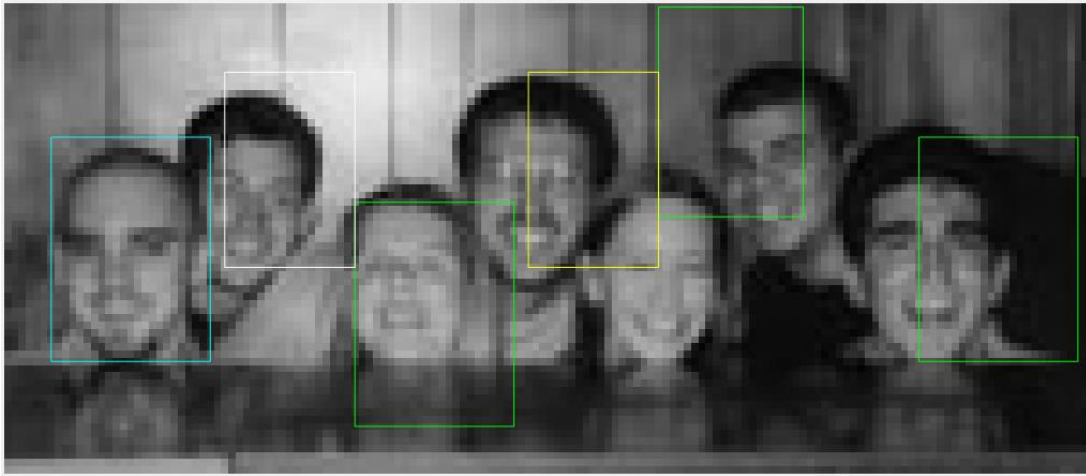


Figure 7.4a. im1.jpg (Recognition rate is 57.1%)

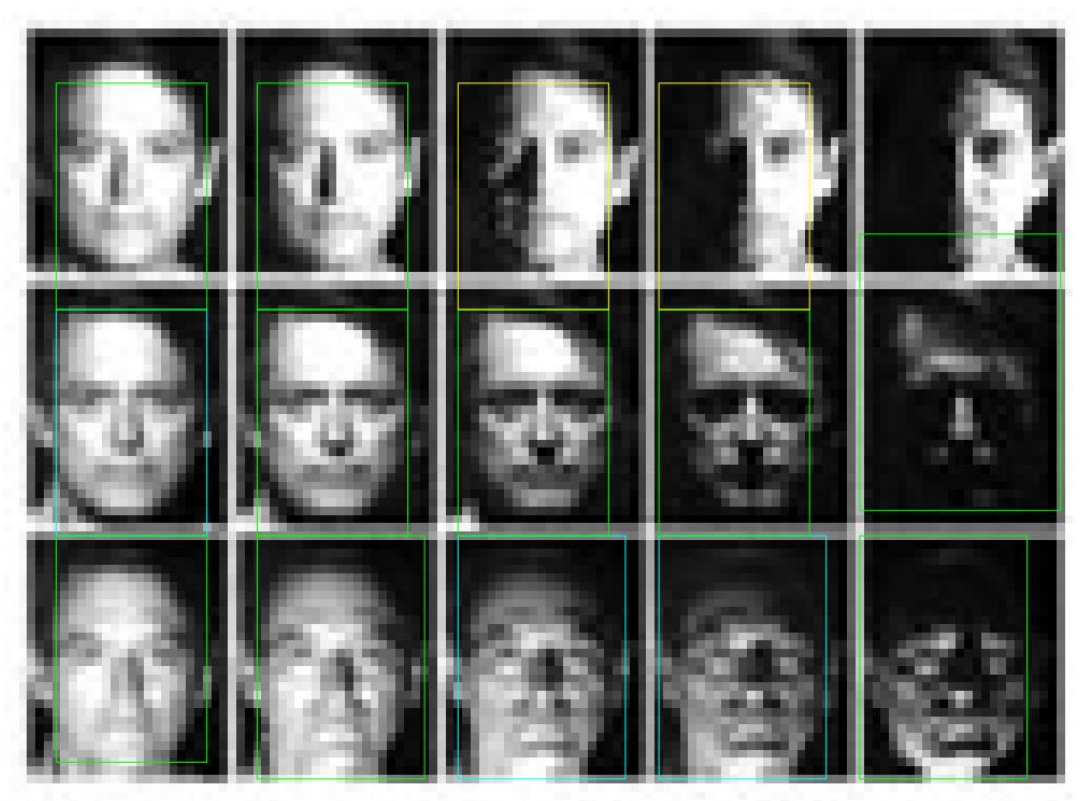


Figure 7.4b. im2.jpg (Recognition rate is 93.3%)

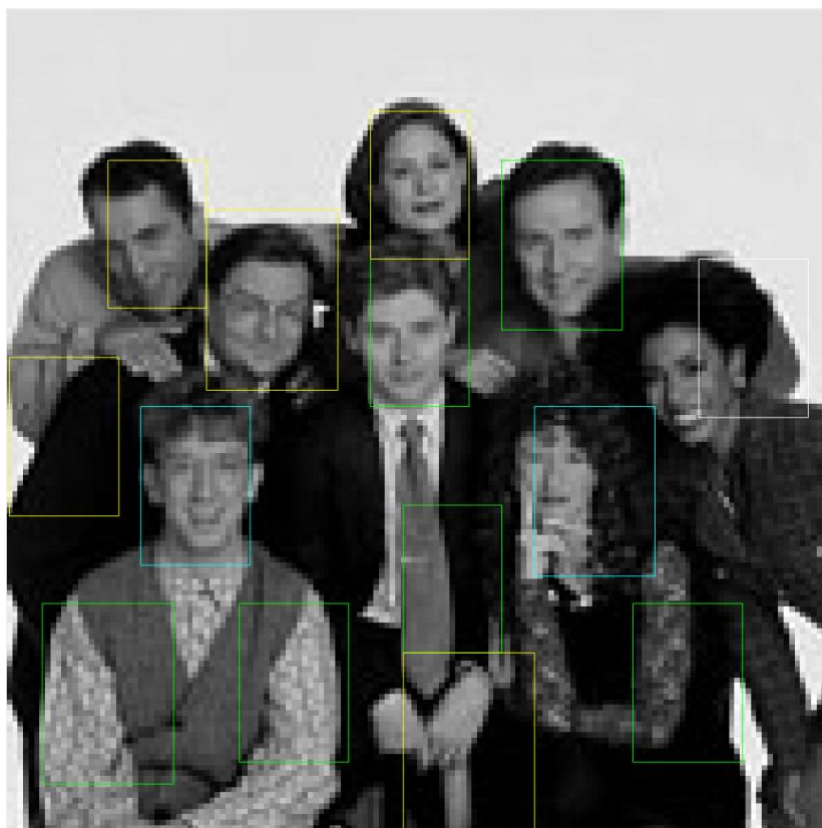


Figure 7.4c. im3.jpg (Recognition rate is 81.25%)

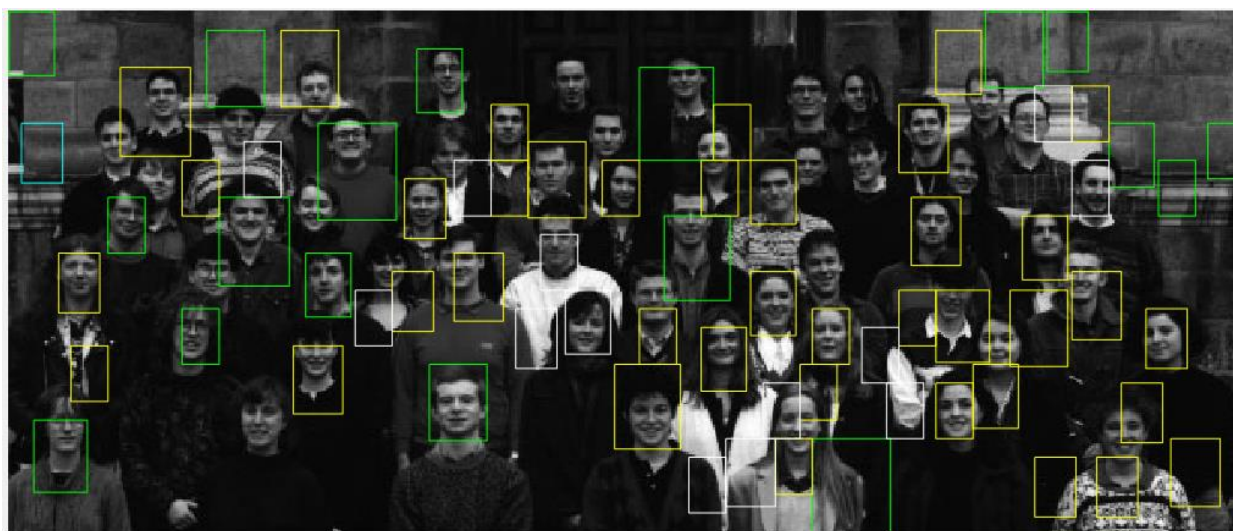


Figure 7.4d. im4.jpg (Recognition rate is 50.9%)

The average recognition rate for this model is 70.6%.

**7.5 Highest Accuracy at 92.50%: HOG → KNN = 2, Histogram Equalisation on,
Trained on 'face_all.cdaset'**



Figure 7.5a. im1.jpg (Recognition rate is 68.75%)

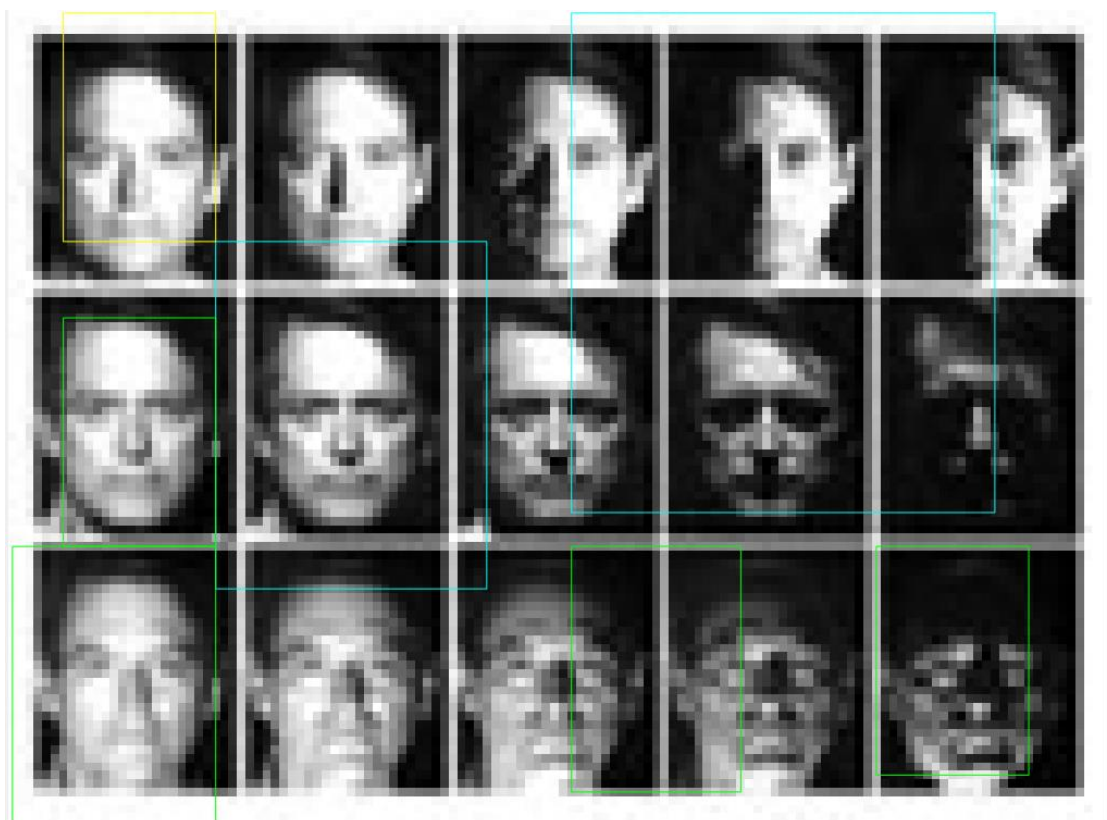


Figure 7.5b. im2.jpg (Recognition rate is 36.7%)

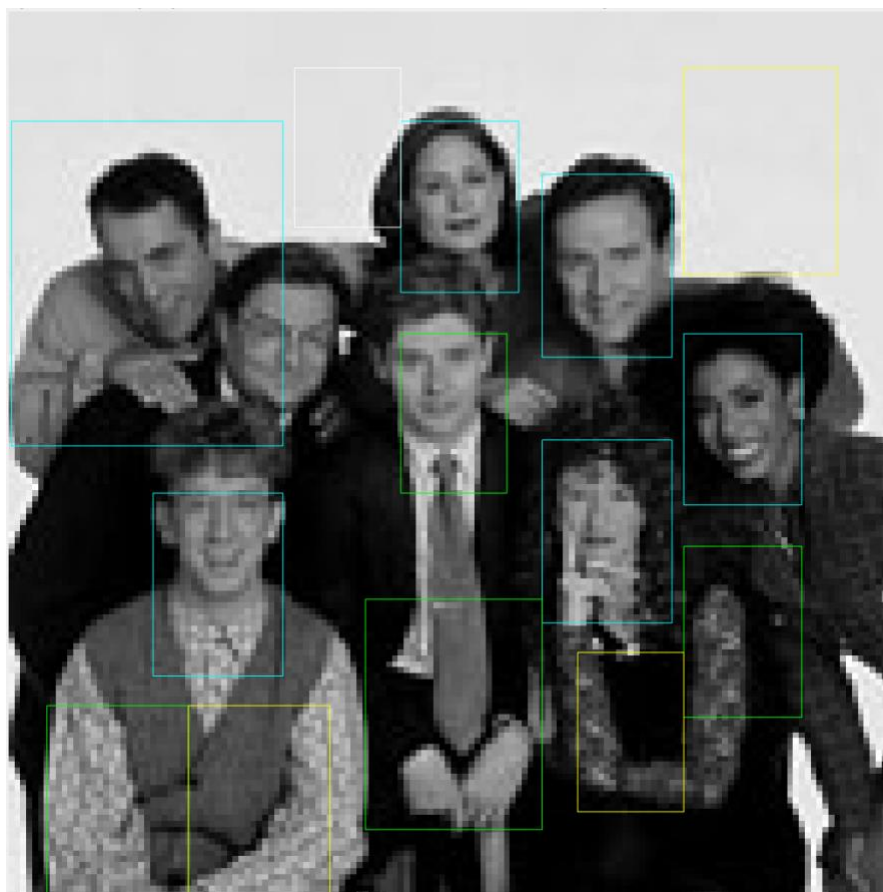


Figure 7.5c. im3.jpg (Recognition rate is 81.25%)

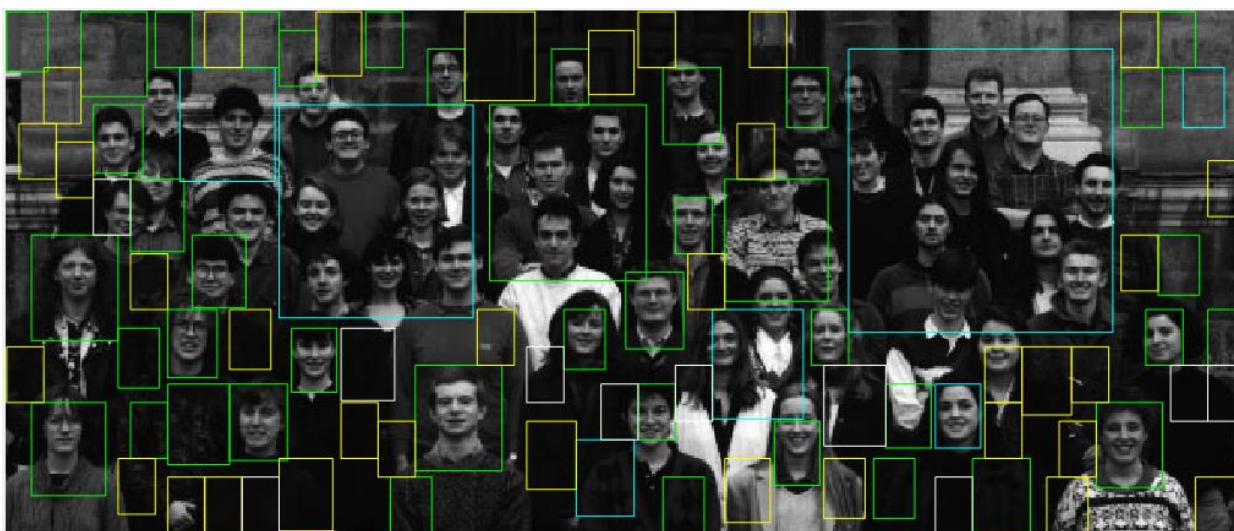


Figure 7.5d. im4.jpg (Recognition rate is 35.3%)

The average recognition rate of this model is 55.5%.

7.6 Best Intuitively based on Random Testing: Full Image → MLP, 15000 neurons, 30 epochs, Histogram Equalisation off, Trained on 'face_all.cdaset'

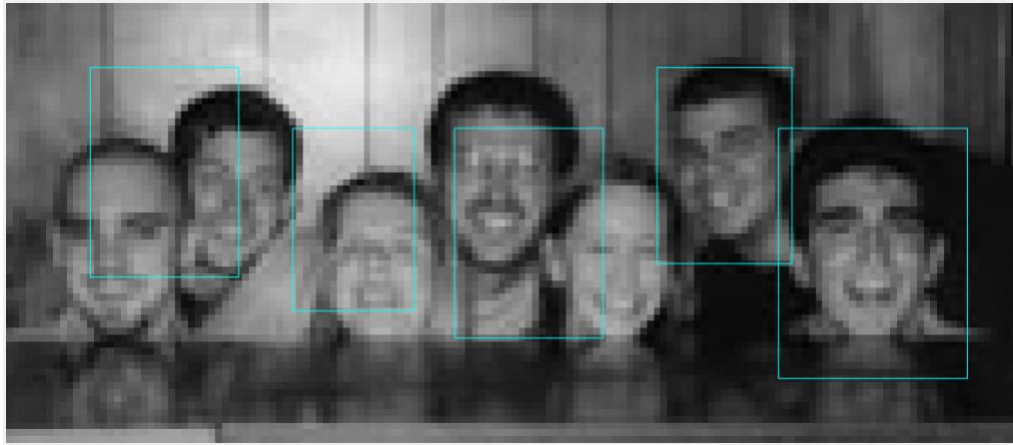


Figure 7.6a. im1.jpg (Recognition rate is 64.3%)

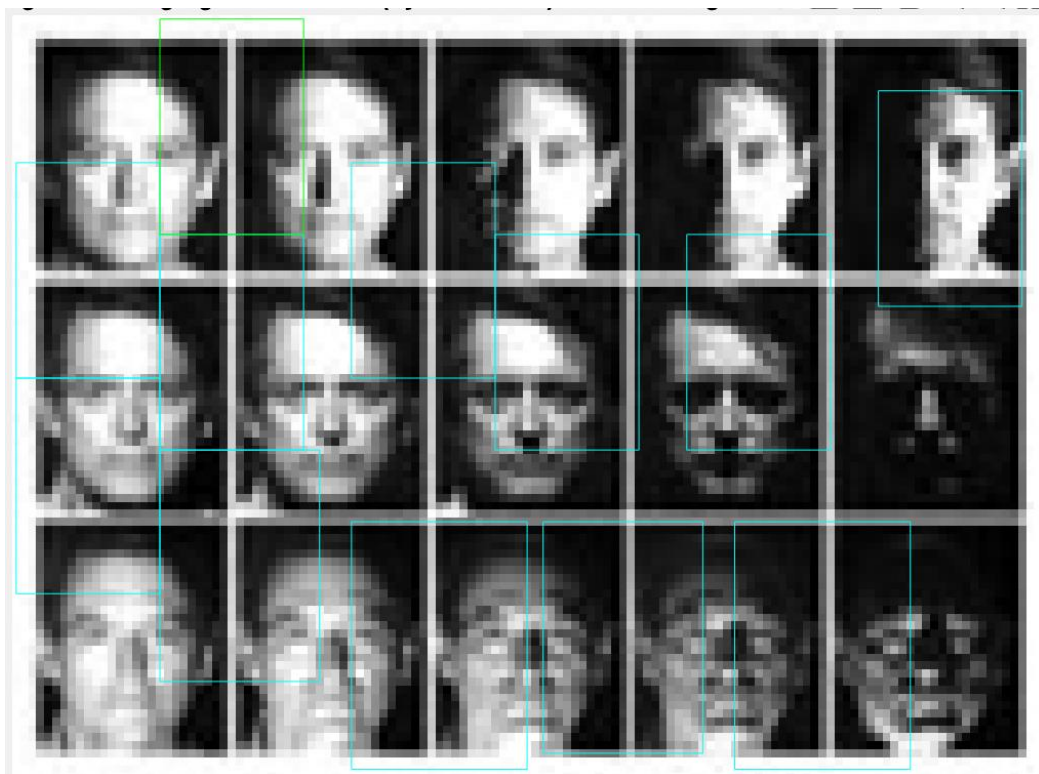


Figure 7.6b. im2.jpg (Recognition rate is 30%)



Figure 7.6c. im3.jpg (Recognition rate is 87.5%)

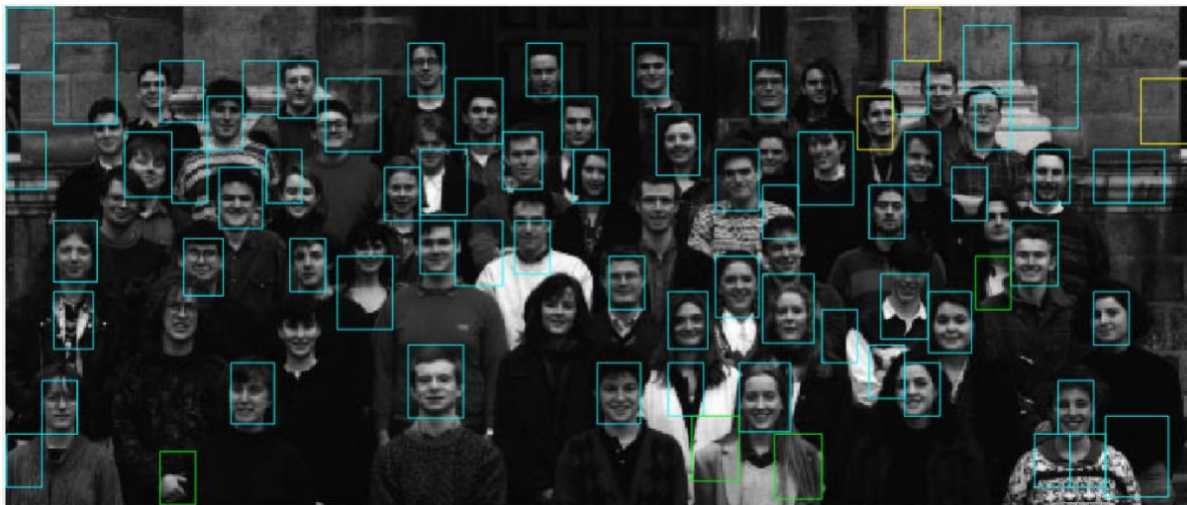


Figure 7.6d. im4.jpg (Recognition rate is 70.7%)

The average recognition rate for this model is 63.1%.

7.7 Recognition Rates Across Models in Section 7.1 to 7.6

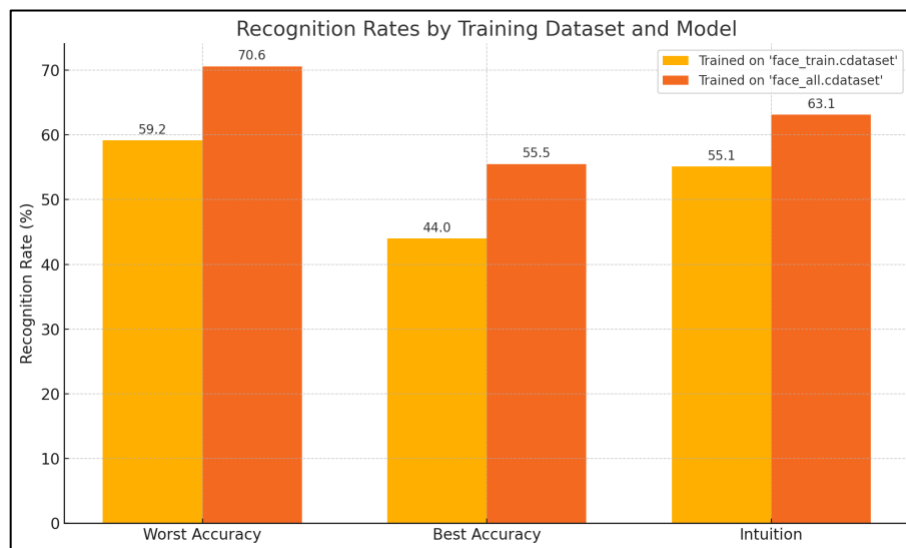


Figure 7.8a. Plots of the data from the 3 models across section 7.1 to 7.6.

An observation that is consistent with the prediction made in Section 6.2 is that models trained on the full dataset (`face_all.cdataset`) performed better than the same models trained only on 70% of the data (`face_train.cdataset`).

It is interesting to note that despite having the best accuracy from cross-validation in the original 70/30 split, the model performed the worst in terms of recognition rate. Surprisingly, the model performed much worse than the model that had the worst accuracy during cross-validation. It is uncertain why, but this could purely be due to variance as a result of external factors in a large detector image that are outside of the controlled testing conditions.

The intuition model was picked based on the performance it had exclusively on 'im4.jpg', where a lot of the detections were clean and there were much less FPs. It is hence important to consider again that other metrics such as these are not taken into account for this evaluation in Section 7, which may have been useful.

'im{1,2}.jpg' both worked the best with the Full Image Raw Pixels → 1NN model, and 'im{3,4}.jpg' both worked the best with the Full Image → MLP model. Since 'im{1,2}.jpg' are smaller images, and 'im{3,4}.jpg' are larger images, this suggests that simpler classifiers like 1-NN perform better with smaller images, while more complex models like an MLP with 15,000 neurons are better suited for larger images.

8. Conclusion

It is interesting to note that in this project, full image raw pixels did surprisingly well (albeit not better than after being PCA reduced to 200 dimensions, or HOG) despite its large feature size and overall lack of processing. Both PCA-200 and HOG seemed to work better because both techniques reduced the large feature vector containing probably redundant information, into something more focused for the machine to learn from. Gabor was found to either be difficult to work with or unsuitable for face detections.

However, the cross-validation accuracy metrics seem to become irrelevant when dealing with small detector images, where the worst models actually did the best. It is uncertain if there is a causality between these two things, but our brief study seems to reveal so.

In general, we would conclude that cross-validation accuracies are only somewhat proportional to actual detection recognition rates, likely due to variables that are outside of the controlled testing environment, for example: object pose, occlusion, clutter, camera angle, lighting and intra-class appearance.

Intra-class appearance is especially relevant in this project, in the sense of skin tones, which greatly affect pixel intensities. This problem is worth mentioning due to the potential of it being able to be tackled, as compared to other elements that are not so controllable. Features that measure relative intensities of neighbouring segments would perform well in tackling this problem of intra-class appearances, for example, Haar features or HOG feature (since even on a dark skin toned person, regions like the eyes should still be darker than their surrounding areas, etc.)

In the future, our go-to model would be the Full Image → PCA-200 → KNN for general purpose face detections, due to the cross-validation accuracies produced in general and the average detector recognition rates.

9. Appendix

Git repository is available at: <https://gitlab.eeecs.qub.ac.uk/CSC3067-2425/CSC3067-2425-G23>

9.1 Verification of Workings in Feature Extraction

To verify that the pipeline/code was working (e.g. features are extracted properly), here are visualisations made within each feature extraction. The functional code to show these visualisations are PCA → SVM model, and in the KNN classifier version for each feature descriptor, in the Git repository.

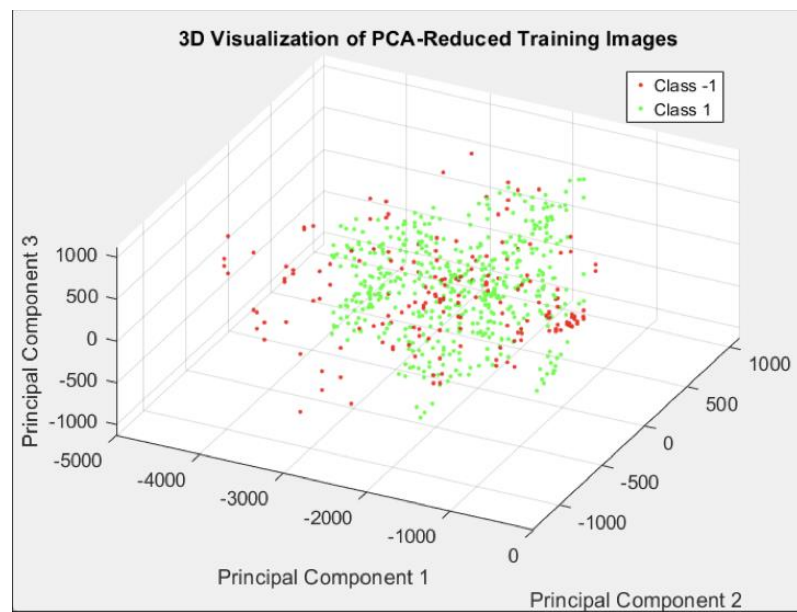


Figure 9.1a. Visualising PCA down to 3 dimensions, between the 2 classes.

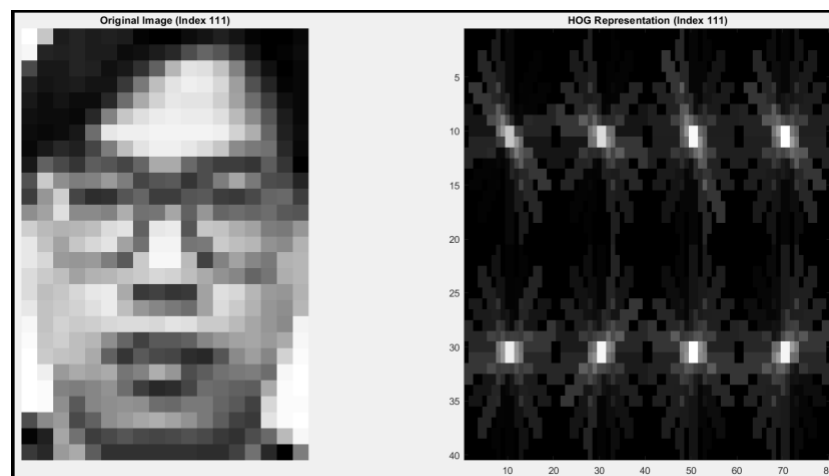


Figure 9.1b. Visualizing HOG features for a given image in the dataset.

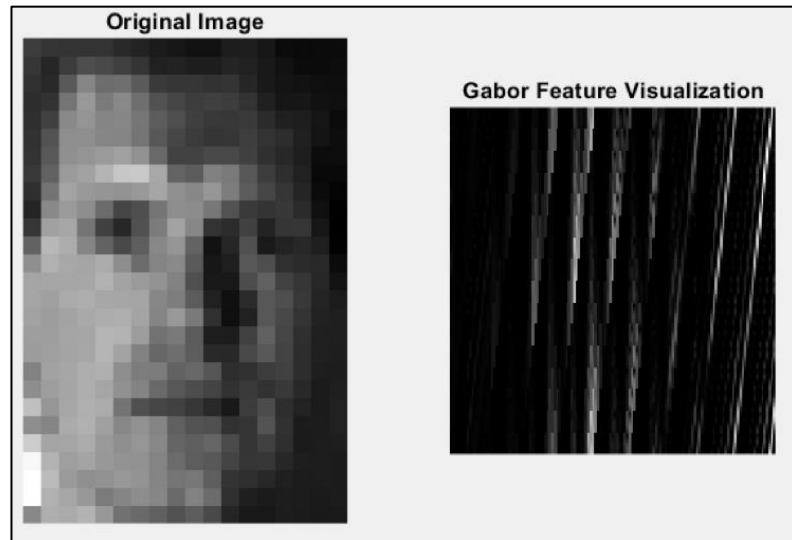


Figure 9.1c. Visualising Gabor features for a given image in the dataset.

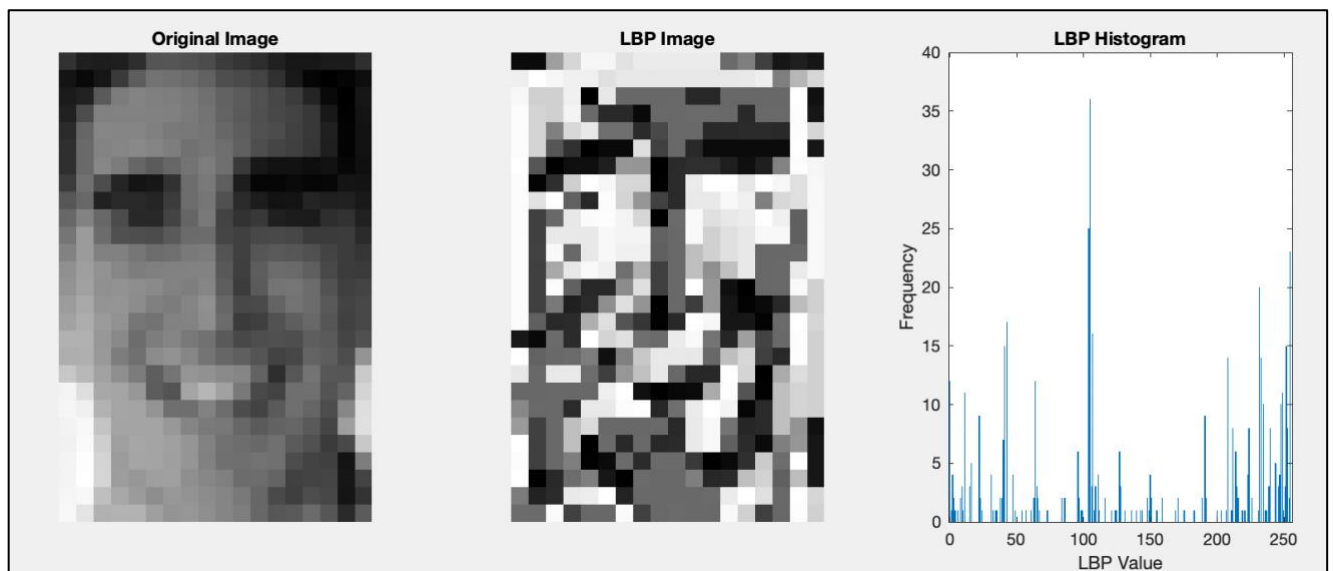


Figure 9.1d. Visualising LBP features for a given image in the dataset.

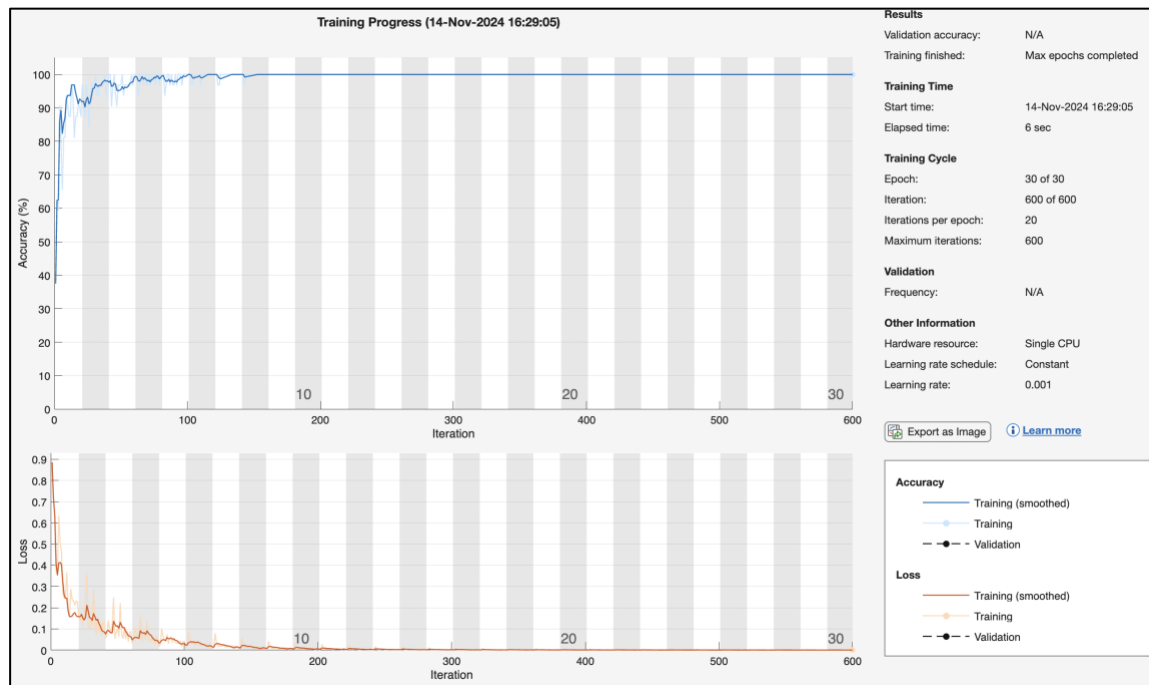


Figure 9.1d. Window that appears in MATLAB during MLP training. Note the rise in accuracy (but this is training accuracy, not the cross-validation accuracy on the test dataset), and plateauing around 100% by the 8th epoch.

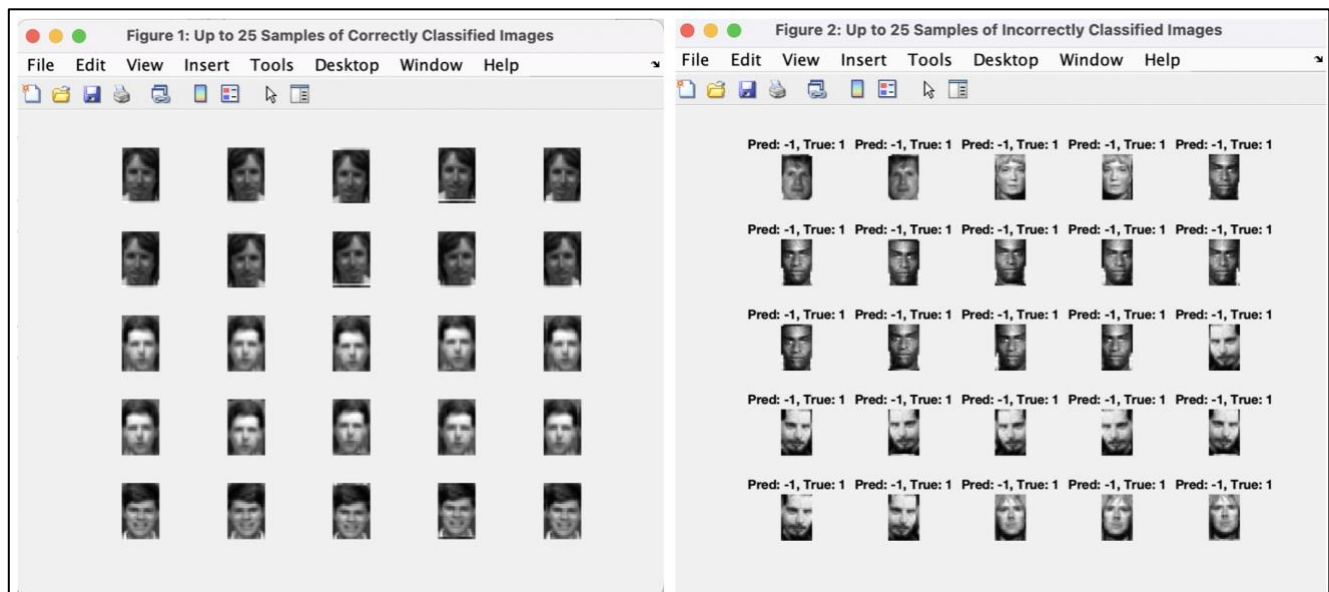


Figure 9.1e. During cross-validation, samples of correctly and incorrectly classified images can be previewed for all models (set to 25 due for convenience). Note that “Pred” means the predicted label in the window of incorrectly classified images, and “True” was the true label.

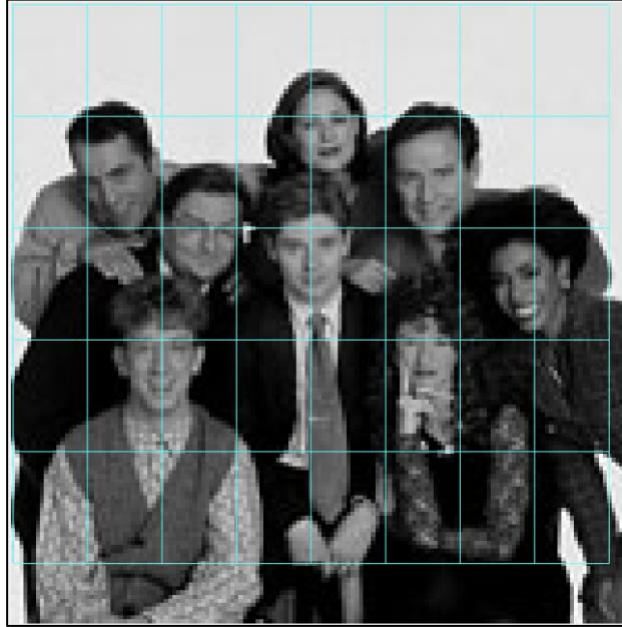


Figure 9.1f. Typical output from detection phase for one of the invalid models. As can be seen, it registers a detection at every possible position. Note how in conjunction with NMS, the detection boxes are perfectly aligned as it removes all overlaps.

```

FullImage_PCA_KNN_LOOCV.m  x  +
217
218 %% Step 3: LOOCV for K-NN with Tuning K
219
220 maxK = 100; % Maximum value of K to check (adjust as needed)
221 totalSamples = size(imagesPCA, 1);
222 accuracyPerK = zeros(maxK, 1); % To store average accuracy for each K
223
224 fprintf('Running LOOCV for K values from 1 to %d:\n', maxK);
225
226 % For each K within the selected range
227 for K = 1:maxK
228     correctPredictions = 0; % Increment the num of correct predictions for this K if it predicts correctly
229
230     % For X number of samples available (to form X number of folds - in this case it's 910 if data augmented)
231     for testIndex = 1:totalSamples
232         % Leave the current sample out as the test instance
233         testFeature = imagesPCA(testIndex, :);
234         testLabel = labels(testIndex);
235
236         % Use remaining samples as the training set
237         trainFeatures = imagesPCA([1:testIndex-1, testIndex+1:end], :);
238         trainLabels = labels([1:testIndex-1, testIndex+1:end]);
239
240         % Initialize KNN model with current training set
241         modelKNN.neighbours = trainFeatures;
242         modelKNN.labels = trainLabels;
243
244         % Perform KNN classification on the test instance with current K
245         predictedLabel = KNNTesting(testFeature, modelKNN, K);
246
247         % Check if the prediction is correct
248         if predictedLabel == testLabel
249             correctPredictions = correctPredictions + 1;
250         end
251     end
252
253     % Calculate and store average accuracy for this K
254     accuracyPerK(K) = correctPredictions / totalSamples;
255     fprintf('K = %d, Average LOOCV Accuracy = %.2f%%\n', K, accuracyPerK(K) * 100);
256 end
257
258 %% Step 4: Determine the Best K
259
260 [bestAccuracy, bestK] = max(accuracyPerK);
261 fprintf('\nBest K = %d with Average LOOCV Accuracy = %.2f%%\n', bestK, bestAccuracy * 100);

```

Figure 9.1g. MATLAB code to test for LOOCV using 'face_all.cdataset'.

```
Running LOOCV for K values from 1 to 100:
K = 1, Average LOOCV Accuracy = 98.46%
K = 2, Average LOOCV Accuracy = 98.68%
K = 3, Average LOOCV Accuracy = 95.82%
K = 4, Average LOOCV Accuracy = 96.48%
K = 5, Average LOOCV Accuracy = 95.16%
K = 6, Average LOOCV Accuracy = 95.38%
K = 7, Average LOOCV Accuracy = 94.51%
K = 8, Average LOOCV Accuracy = 94.51%
K = 9, Average LOOCV Accuracy = 93.41%
K = 10, Average LOOCV Accuracy = 94.29%
K = 11, Average LOOCV Accuracy = 92.31%
K = 12, Average LOOCV Accuracy = 92.75%
K = 13, Average LOOCV Accuracy = 92.09%

⋮

K = 95, Average LOOCV Accuracy = 81.10%
K = 96, Average LOOCV Accuracy = 81.10%
K = 97, Average LOOCV Accuracy = 81.10%
K = 98, Average LOOCV Accuracy = 80.88%
K = 99, Average LOOCV Accuracy = 81.10%
K = 100, Average LOOCV Accuracy = 81.10%

Best K = 2 with Average LOOCV Accuracy = 98.68%
```

Figure 9.1h. MATLAB console output for LOOCV for each K, then showing the K with the best average LOOCV accuracy.